Artificial Intelligence Concepts and Machine Learning Techniques (AI – 103)

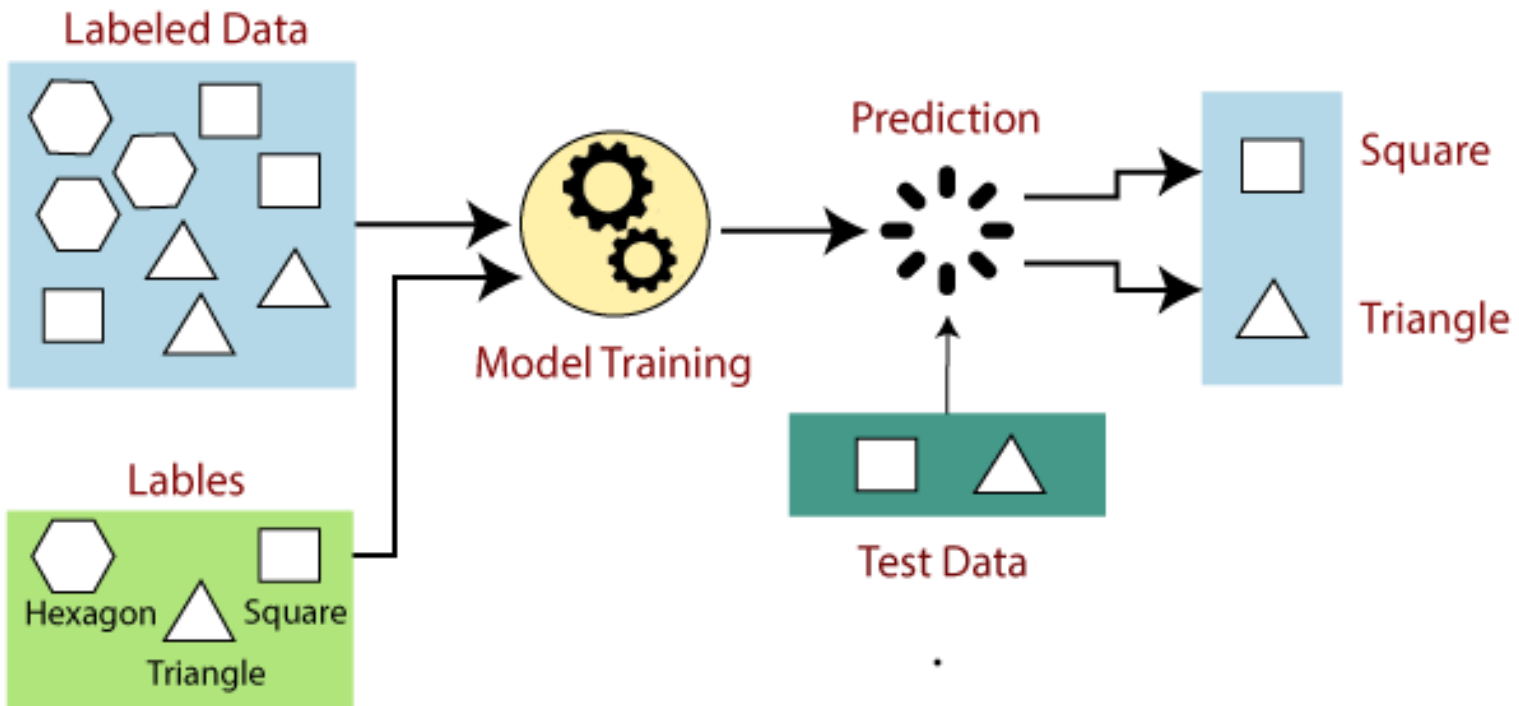
Walaa H. Elashmawi

Professor of AI

walaa.hassan@miuegypt.edu.eg

Academic Profiles: Google Scholar Scopus ResearchGate LinkedIn

Supervised Machine Learning Models


Labeled Data
Lables
Hexagon
Square
Triangle
Model Training
Prediction
Test Data
Square
Triangle

Support Vector Machine

What is Support Vector Machine (SVM)

SVM is a supervised machine learning algorithm which is mainly used to classify data into different classes. It trains on a set of labelled data.

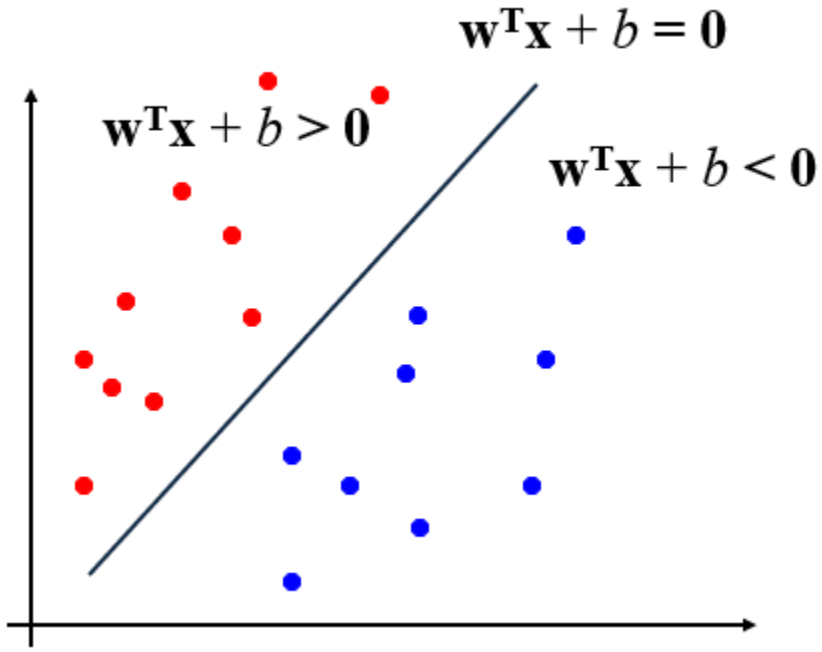
Unlike most algorithms, SVM makes use of a hyperplane which acts like a decision boundary between the various classes.

SVM studies the labelled training data and then classifies any new input data depending on what it learned in the training phase.

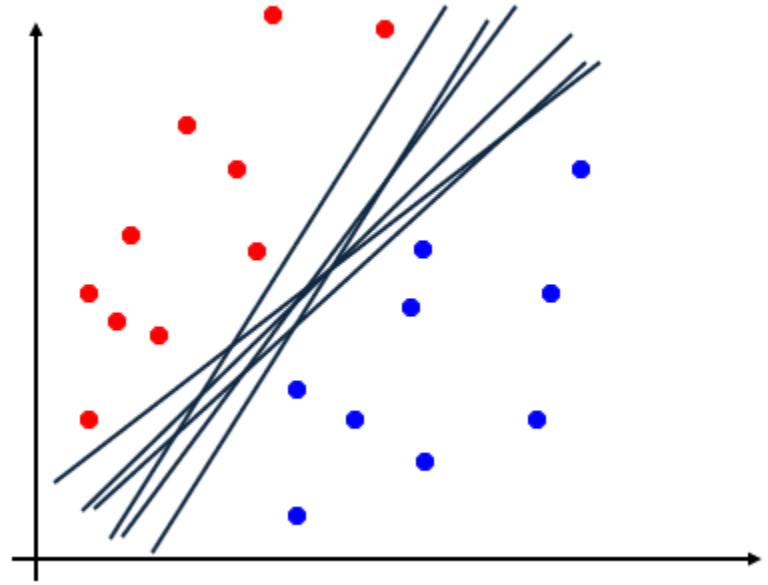
SVM is used for classifying non-linear data by using the ‘kernel trick’.

Linear Separators

- Binary classification can be viewed as the task of separating classes in feature space:



w: decision hyperplane normal vector
$\mathbf{x}_i$: data point *i*



Classifier: $f(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b)$

Which of the linear separators is optimal?

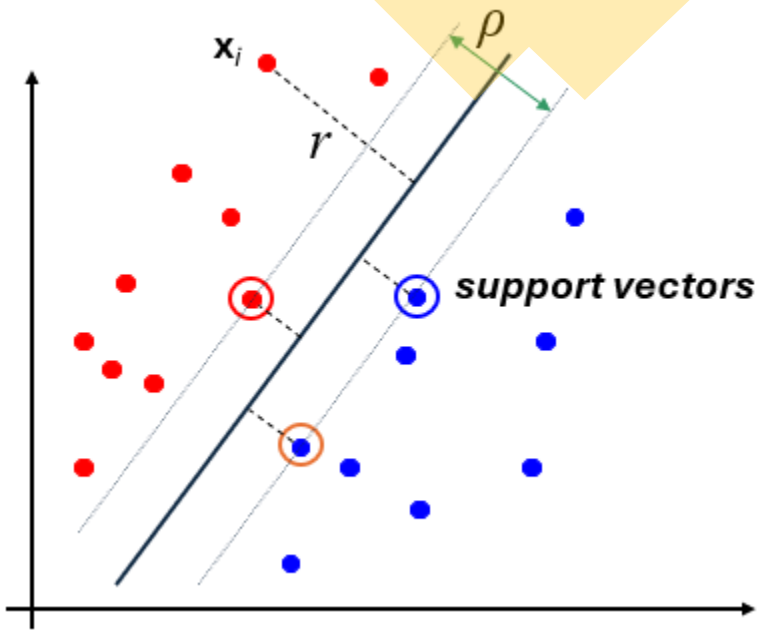
Classification Margin

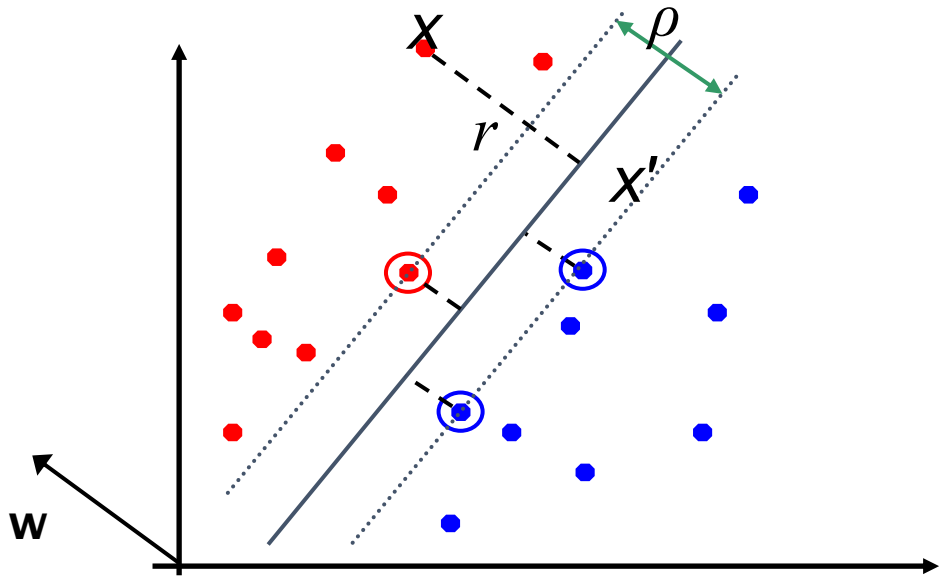
- Distance from example $\mathbf{x}_i$ to the separator is

$$r = \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|}$$

- Examples closest to the hyperplane are ***support vectors***.

- ***Margin*** ρ of the separator is the distance between support vectors.





Derivation of finding *r*:

- Dotted line $\mathbf{x}'-\mathbf{x}$ is perpendicular to decision boundary so parallel to $\mathbf{w}$.
- Unit vector is $\mathbf{w}/|\mathbf{w}|$, so line is $r\mathbf{w}/|\mathbf{w}|$.
  $\mathbf{x}' = \mathbf{x} - yr\mathbf{w}/|\mathbf{w}|$ and satisfies $\mathbf{w}^T\mathbf{x}' + b = 0$
  So $\mathbf{w}^T(\mathbf{x} - yr\mathbf{w}/|\mathbf{w}|) + b = 0$
  Recall that $|\mathbf{w}| = \sqrt{\mathbf{w}^T\mathbf{w}}$ ➔ $\mathbf{w}^T\mathbf{x} - yr|\mathbf{w}| + b = 0$
  So, solving for r gives $r = y(\mathbf{w}^T\mathbf{x} + b)/|\mathbf{w}|$

Linear SVM Mathematically

- Assume that all data is at least distance 1 from the hyperplane, then the following two constraints follow for a training set $\{(\mathbf{x_i}, y_i)\}$

$$\mathbf{w^T x_i} + b \geq 1 \quad \text{if } y_i = 1$$

$$\mathbf{w^T x_i} + b \leq -1 \quad \text{if } y_i = -1$$

The linearly separable case

- For support vectors, the inequality becomes an equality
- Then, since each example's distance from the hyperplane is $r = y\dfrac{\mathbf{w}^T\mathbf{x} + b}{\|\mathbf{w}\|}$
- The margin is: $\rho = \dfrac{2}{\|\mathbf{w}\|}$

Linear Support Vector Machine (SVM)

- **Hyperplane**

  $\mathbf{w}^T \mathbf{x} + b = 0$

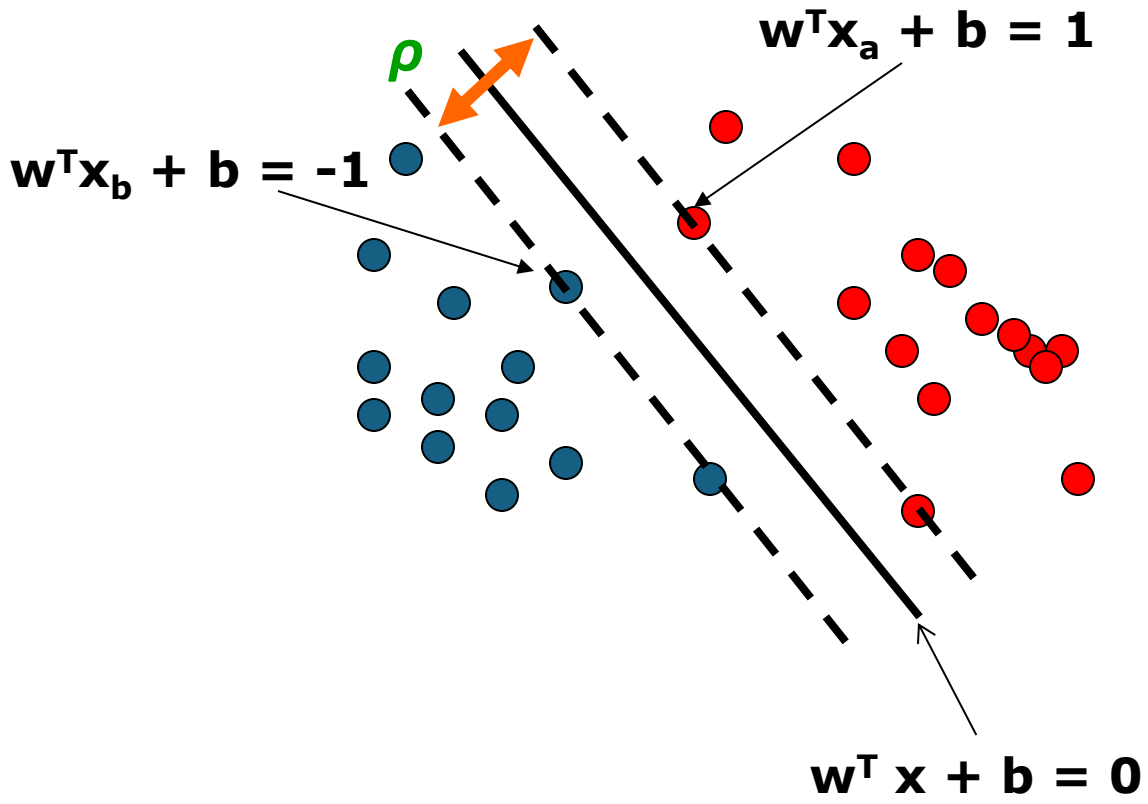
- **Extra scale constraint**:

  $\min_{i=1,\ldots,n} |\mathbf{w}^T \mathbf{x}_i + b| = 1$

- This implies:

  $\mathbf{w}^T(\mathbf{x}_a - \mathbf{x}_b) = 2$

  $\rho = ||\mathbf{x}_a - \mathbf{x}_b||_2 = 2/||\mathbf{w}||_2$



Linear SVMs Mathematically (cont.)

- Then we can formulate the *quadratic optimization problem:*

Find $\mathbf{w}$ and b such that

$$\rho = \frac{2}{\|\mathbf{w}\|} \text{ is maximized}$$

and for all $(\mathbf{x}_i, y_i)$, $i=1..n$: $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

- Which can be reformulated as:

Find $\mathbf{w}$ and b such that

$\mathbf{\Phi}(\mathbf{w}) = \|\mathbf{w}\|^2 = \mathbf{w}^T\mathbf{w}$ is minimized

and for all $(\mathbf{x}_i, y_i)$, $i=1..n$: $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

Solving the Optimization Problem

Find $\mathbf{w}$ and $\boldsymbol{b}$ such that:

$$\Phi(\mathbf{w}) = \mathbf{w}^T\mathbf{w} \text{ is minimized}$$

and for all $(\mathbf{x}_i, y_i)$, $i=1...n$: $y_i (\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

- Need to optimize a *quadratic* function subject to *linear* constraints.
- Quadratic optimization problems are a well-known class of mathematical programming problems for which several (non-trivial) algorithms exist.
- The solution involves constructing a *dual problem* where a *Lagrange multiplier* α_i is associated with every inequality constraint in the primal (original) problem:

Find $\alpha_1 ... \alpha_n$ such that:

$\mathbf{Q}(\boldsymbol{\alpha}) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$ is maximized and

(1) $\Sigma\alpha_i y_i = 0$

(2) $\alpha_i \geq 0$ for all α_i

The Optimization Problem Solution

- Given a solution $\alpha_1 \ldots \alpha_n$ to the dual problem, solution to the primal is:

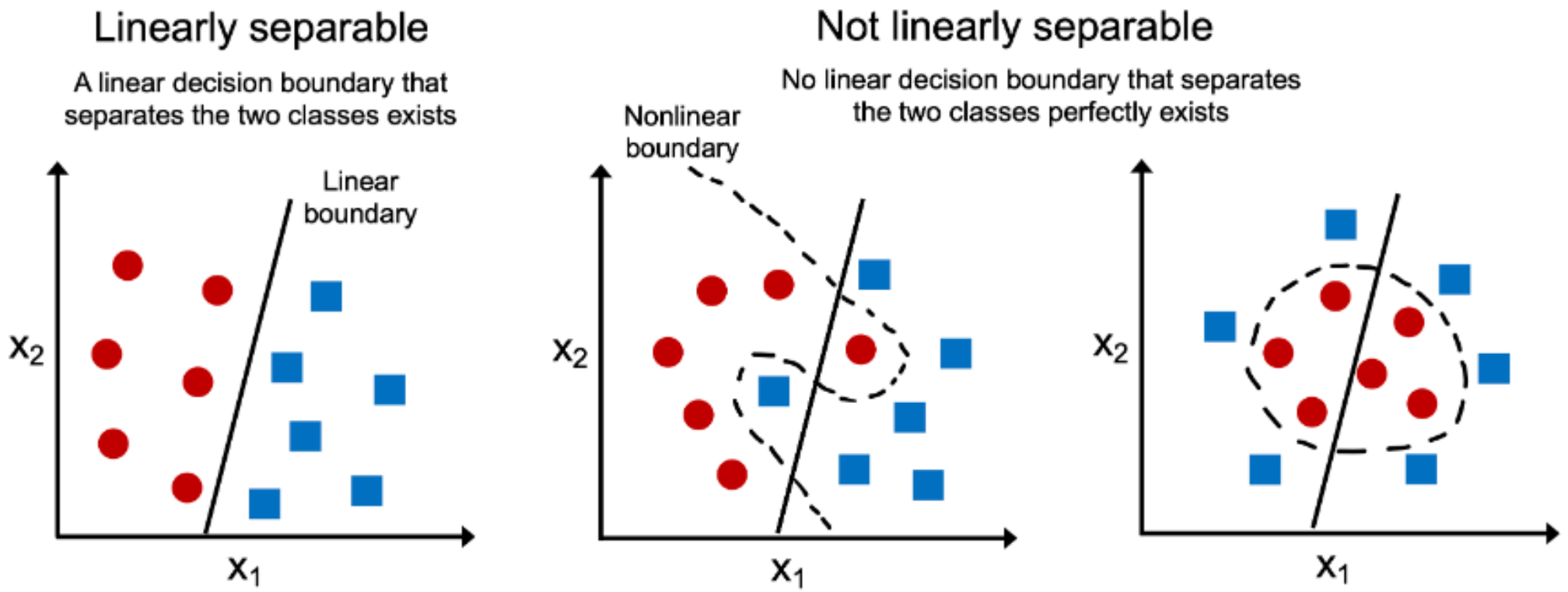
$$\mathbf{w} = \Sigma \alpha_i y_i \mathbf{x}_i \qquad b = y_k - \Sigma \alpha_i y_i \mathbf{x}_i{}^{\mathrm{T}} \mathbf{x}_k \quad \text{for any } \alpha_k > 0$$

- Each non-zero α_i indicates that corresponding $\mathbf{x}_i$ is a support vector.
- Then the classifying function is (note that we don't need **w** explicitly):

$$f(\mathbf{x}) = \Sigma \alpha_i y_i \mathbf{x}_i{}^{\mathrm{T}} \mathbf{x} + b$$

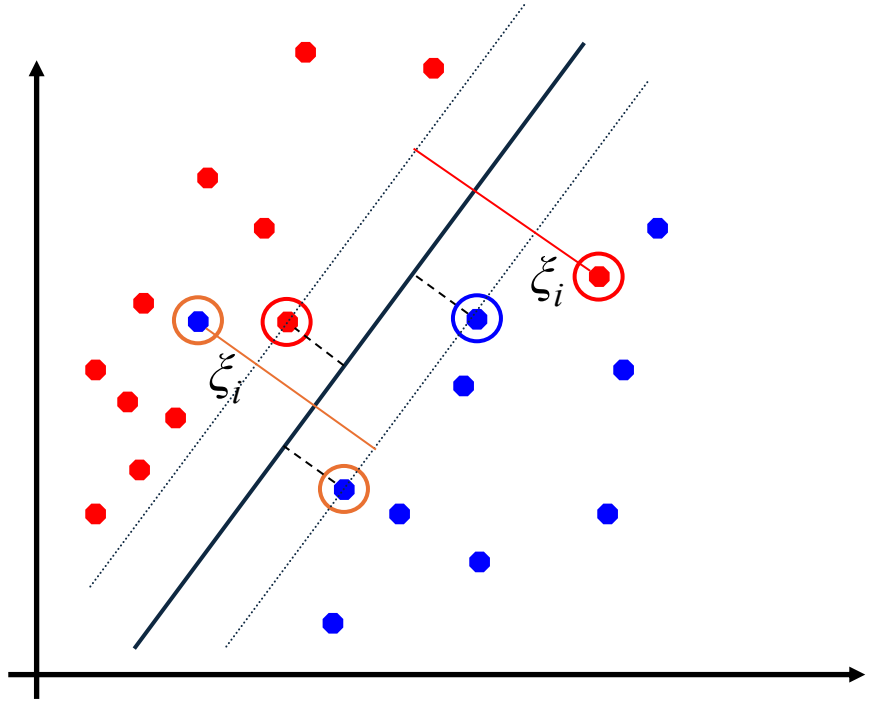
- Notice that it relies on an *inner product* between the test point **x** and the support vectors $\mathbf{x}_i$ – we will return to this later.
- Also keep in mind that solving the optimization problem involved computing the inner products $\mathbf{x}_i{}^{\mathrm{T}} \mathbf{x}_j$ between all training points.

Linear vs. Nonlinear



Soft Margin Classification

- What if the training set is not linearly separable?
- *Slack variables* ξ_i can be added to allow misclassification of difficult or noisy examples, resulting margin called *soft*.



Soft Margin Classification Mathematically

- The old formulation:

Find $\mathbf{w}$ and b such that
$\mathbf{\Phi}(\mathbf{w}) = \mathbf{w}^T\mathbf{w}$ is minimized
and for all $(\mathbf{x}_i, y_i)$, $i=1..n$: $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$

- Modified formulation incorporates slack variables:

Find $\mathbf{w}$ and b such that
$\mathbf{\Phi}(\mathbf{w}) = \mathbf{w}^T\mathbf{w} + C\Sigma\xi_i$ is minimized
and for all $(\mathbf{x}_i, y_i)$, $i=1..n$: $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$

- Parameter C can be viewed as a way to control overfitting:

It "trades off" the relative importance of maximizing the margin and fitting the training data. It acts as a regularization term

Soft Margin Classification – Solution

- Dual problem is identical to separable case (would *not* be identical if the 2-norm penalty for slack variables $C\Sigma\xi_i^2$ was used in primal objective, we would need additional Lagrange multipliers for slack variables):

Find $\alpha_1 \ldots \alpha_N$ such that
$\mathbf{Q}(\boldsymbol{\alpha}) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$ is maximized and
(1) $\Sigma\alpha_i y_i = 0$
(2) $0 \leq \alpha_i \leq C$ for all α_i

- Again, $\mathbf{x}_i$ with non-zero α_i will be support vectors.
- Solution to the dual problem is:

$\mathbf{w} = \Sigma\alpha_i y_i \mathbf{x}_i$
$b = y_k(1-\xi_k) - \Sigma\alpha_i y_i \mathbf{x}_i^T\mathbf{x}_k$ for any k s.t. $\alpha_k > 0$

Again, we don't need to compute **w** explicitly for classification:

$f(\mathbf{x}) = \Sigma\alpha_i y_i \mathbf{x}_i^T\mathbf{x} + b$

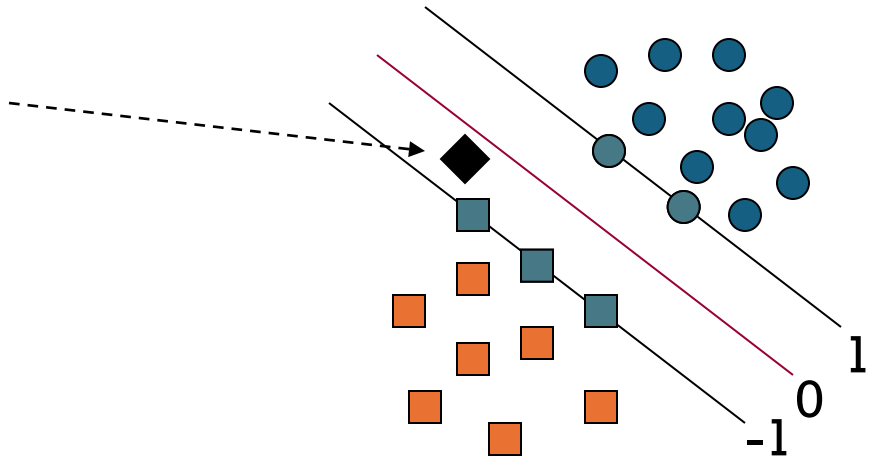
Classification with SVMs

- Given a new point **x**, we can score its projection onto the hyperplane normal:
  - I.e., compute score: $\mathbf{w}^T\mathbf{x} + b = \Sigma \alpha_i y_i \mathbf{x}_i^T\mathbf{x} + b$
    - Decide class based on whether < or > 0
  - Can set confidence threshold *t*.

Score > *t*: yes

Score < -*t*: no

Else: don't know



Linear SVMs: Summary

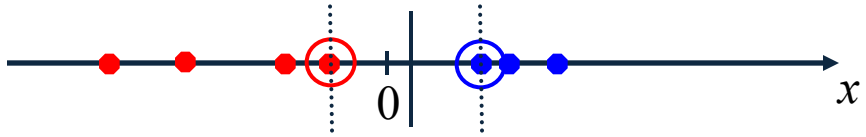
- The classifier is a *separating hyperplane*.
- The most "important" training points are the support vectors; they define the hyperplane.
- Quadratic optimization algorithms can identify which training points $\mathbf{x}_i$ are support vectors with non-zero Lagrangian multipliers α_i.
- Both in the dual formulation of the problem and in the solution, training points appear only inside inner products:

Find $\alpha_1 \ldots \alpha_N$ such that
$\mathbf{Q}(\boldsymbol{\alpha}) = \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j$ is maximized and
(1) $\Sigma\alpha_i y_i = 0$
(2) $0 \le \alpha_i \le C$ for all α_i

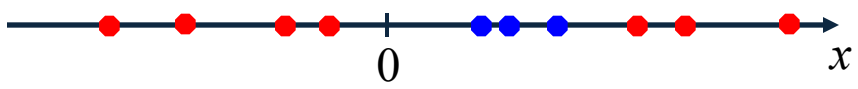
$$f(\mathbf{x}) = \Sigma\alpha_i y_i \mathbf{x}_i^T\mathbf{x} + b$$

Non-linear SVMs

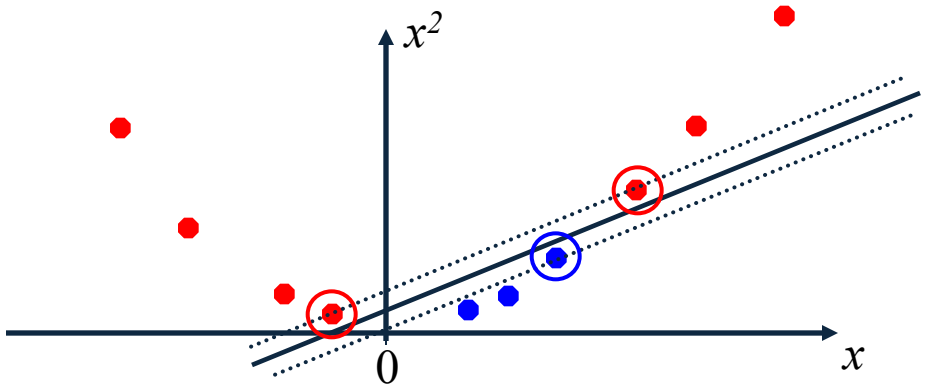
- Datasets that are linearly separable (with some noise) work out great:



- But what are we going to do if the dataset is just too hard?

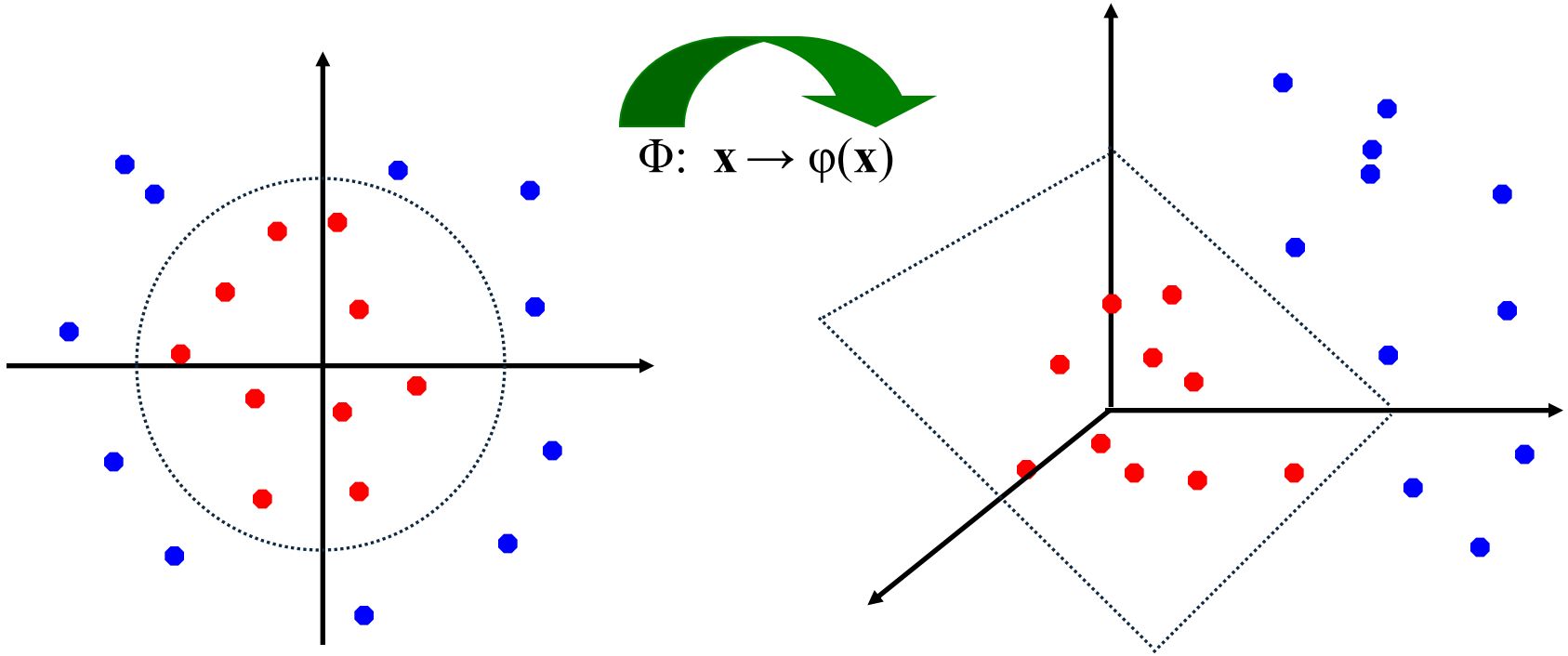


- How about … mapping data to a higher-dimensional space:

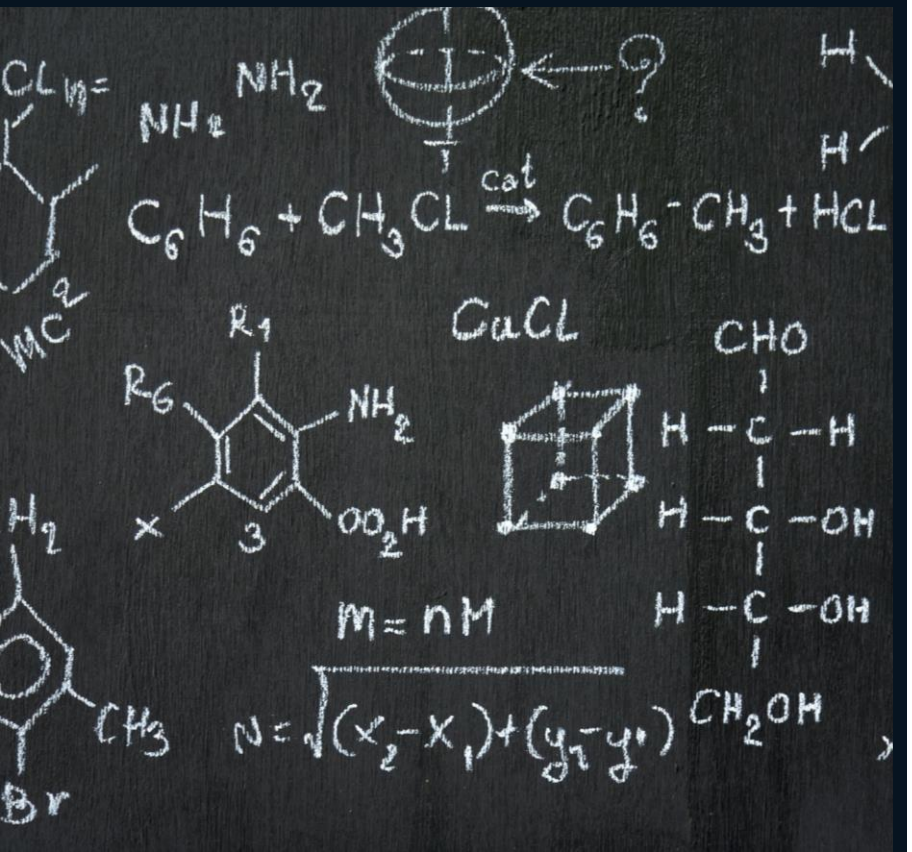


Non-linear SVMs: Feature spaces

- **General idea:** the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:



The “Kernel Trick”



- The linear classifier relies on inner product between vectors $K(\mathbf{x}_i,\mathbf{x}_j)=\mathbf{x}_i^T\mathbf{x}_j$
- If every datapoint is mapped into high-dimensional space via some transformation Φ: $\mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i,\mathbf{x}_j)= \boldsymbol{\phi}(\mathbf{x}_i)^T\boldsymbol{\phi}(\mathbf{x}_j)$$

- A *kernel function* is a function that is equivalent to an inner product in some feature space.

- Example:

  2-dimensional vectors $\mathbf{x}=[x_1\ \ x_2]$; let $K(\mathbf{x}_i,\mathbf{x}_j)=(1 + \mathbf{x}_i^T\mathbf{x}_j)^2$,

  Need to show that $K(\mathbf{x}_i,\mathbf{x}_j)= \boldsymbol{\phi}(\mathbf{x}_i)^T\boldsymbol{\phi}(\mathbf{x}_j)$:

  $K(\mathbf{x}_i,\mathbf{x}_j)=(1 + \mathbf{x}_i^T\mathbf{x}_j)^2 = 1+ x_{i1}^2x_{j1}^2 + 2\, x_{i1}x_{j1}\, x_{i2}x_{j2}+ x_{i2}^2x_{j2}^2 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2}=$

  $= [1\ \ x_{i1}^2\ \sqrt{2}\, x_{i1}x_{i2}\ \ x_{i2}^2\ \sqrt{2}x_{i1}\ \sqrt{2}x_{i2}]^T\,[1\ \ x_{j1}^2\ \sqrt{2}\, x_{j1}x_{j2}\ \ x_{j2}^2\ \sqrt{2}x_{j1}\ \sqrt{2}x_{j2}] =$

  $= \boldsymbol{\phi}(\mathbf{x}_i)^T\boldsymbol{\phi}(\mathbf{x}_j)$, where, $\boldsymbol{\phi}(\mathbf{x}) = [1\ \ x_1^2\ \sqrt{2}\, x_1x_2\ \ x_2^2\ \ \sqrt{2}x_1\ \sqrt{2}x_2]$

- Thus, a kernel function *implicitly* maps data to a high-dimensional space (without the need to compute each $\boldsymbol{\phi}(\mathbf{x})$ explicitly).

Most common Kernel Functions

Kernel	Formula / Expression	Key Parameter(s)	When / Use Case	Notes & Tradeoffs
Linear	$K(x, y) = x \cdot y$	—	Data is (approximately) linearly separable	Fast, simple, interpretable
Polynomial	$K(x, y) = (x \cdot y + c)^d$	c (constant term), d (degree)	When you expect polynomial (nonlinear) relations	High degree → overfitting; low degree → underfitting
RBF	$K(x, y) = \exp(-\gamma \|\|x - y\|\|^2)$	γ (width / influence)	Common default for nonlinear boundaries	Flexible, needs tuning; large γ → overfit
Gaussian	$K(x, y) = \exp(-\|\|x - y\|\|^2 / 2\sigma^2)$	σ (spread)	Equivalent / special case of RBF kernel	Similar behavior to RBF; parameterized differently
Sigmoid	$K(x, y) = \tanh(\gamma x^T y + r)$	γ, r	Inspired by neural networks	Acts like neural net; not always positive definite

Non-linear SVMs Mathematically

- Dual problem formulation:

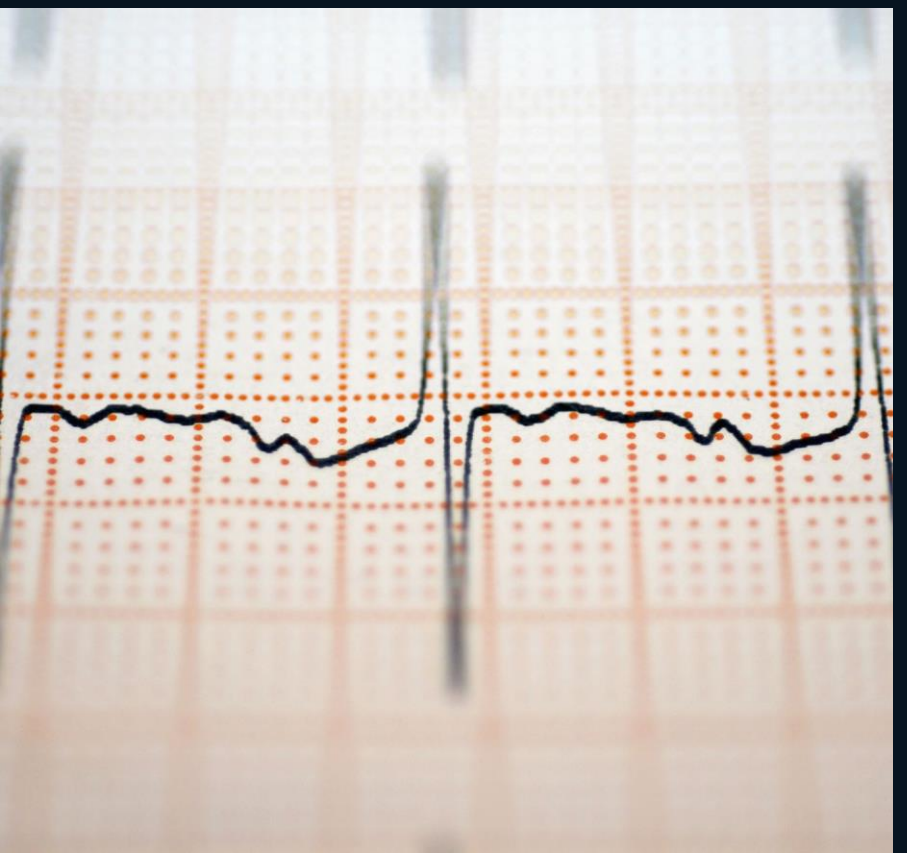
Find $\alpha_1 \ldots \alpha_n$ such that
$\mathbf{Q}(\boldsymbol{\alpha}) = \Sigma \alpha_i - \frac{1}{2}\Sigma\Sigma \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$ is maximized and
(1) $\Sigma \alpha_i y_i = 0$
(2) $\alpha_i \geq 0$ for all α_i

- The solution is:

$f(\mathbf{x}) = \Sigma \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_j) + b$

- Optimization techniques for finding α_i's remain the same!

Why RBF kernel?



- RBF is the main kernel function because of the following reasons:

➢ The RBF kernel nonlinearly maps samples into a higher dimensional space unlike to linear kernel.

➢ The RBF kernel has less hyper parameters than the polynomial kernel.

➢ The RBF kernel has less numerical difficulties.

Model selection of SVM

Model selection is also an important issue in SVM. Its success depends on the tuning of several parameters which affect the generalization error.

If we use the linear SVM, we only need to tune the cost parameter C.

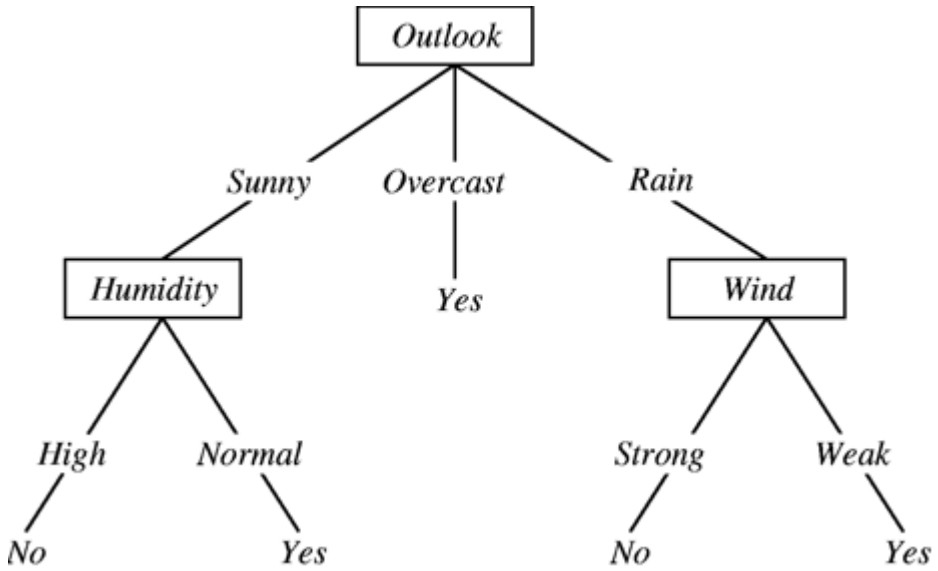
As many problems are non-linearly separable, we need to select the cost parameter C and kernel parameters y, d.

Decision Tree

Decision Tree (DT)

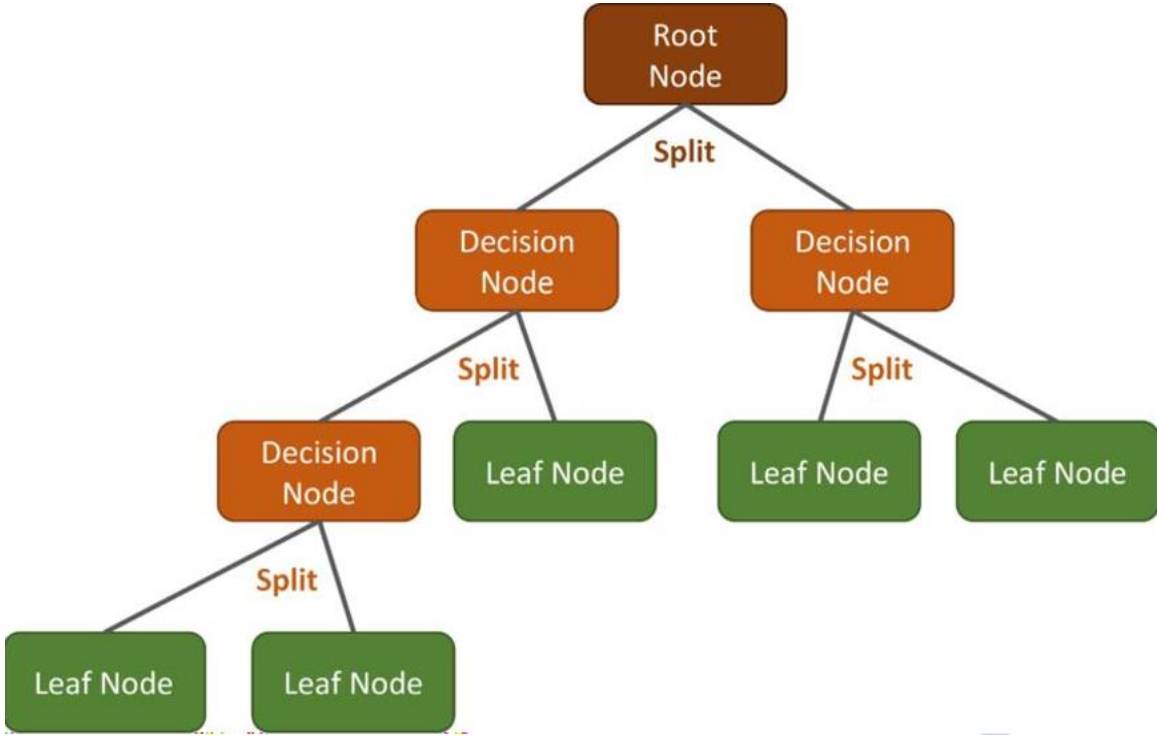
- A decision tree is a popular machine learning algorithm used for both classification and regression tasks. It is a supervised learning algorithm that is particularly useful for its simplicity, interpretability, and ability to handle both categorical and numerical data.

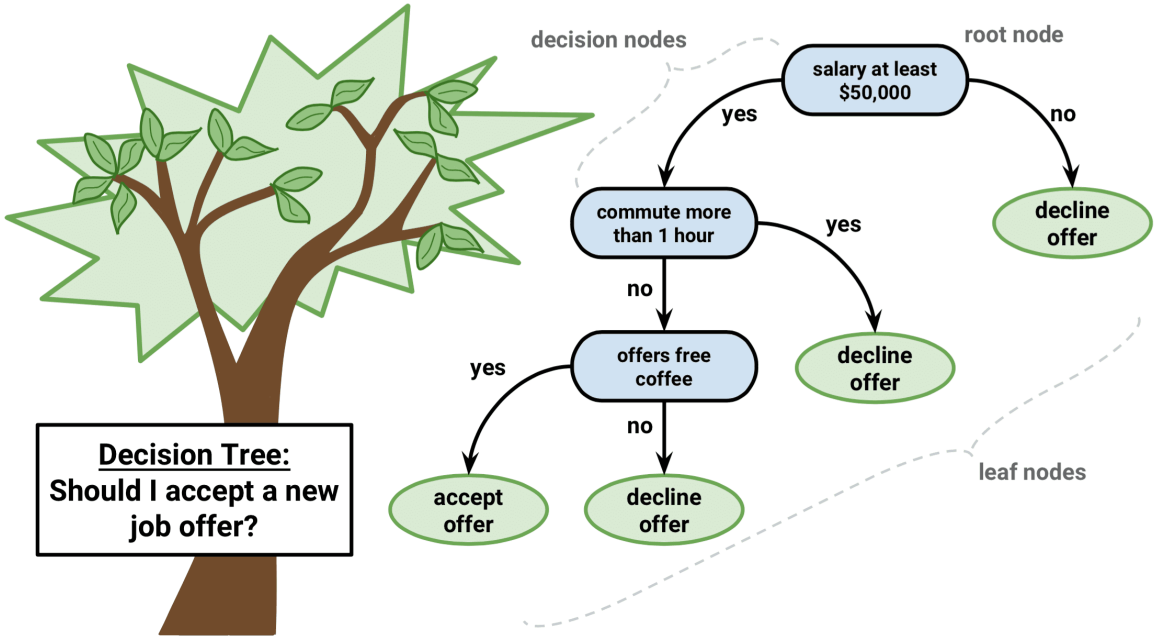
- A decision tree models decisions or predictions as a tree-like structure, where each internal node represents a decision based on a feature (attribute), each branch represents an outcome of that decision, and each leaf node represents a class label (in classification) or a numerical value (in regression).



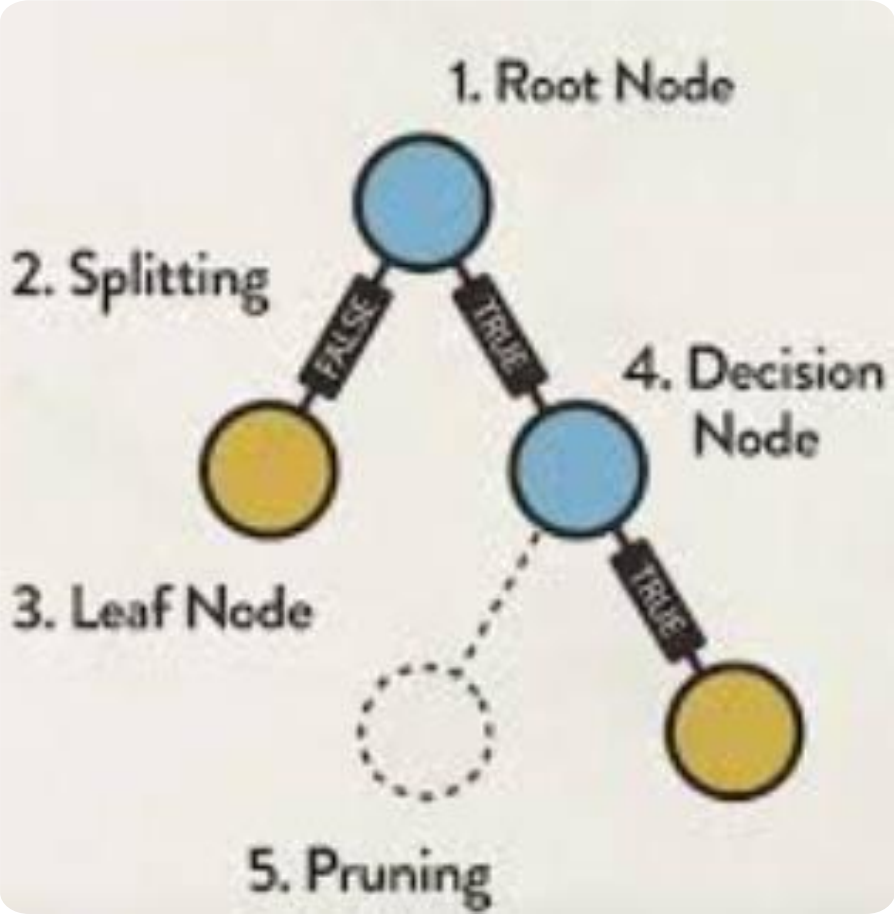
The decision tree structure

- It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.



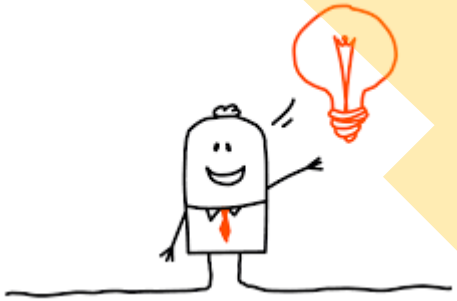


Building Blocks of DT



- **Root Node**: At the top of the tree is the root node, which represents the entire dataset , which further gets divided into two or more homogeneous sets.
- **Splitting:** The algorithm selects the feature that best splits the data into more homogenous subsets. It does this by evaluating various splitting criteria, such as Gini impurity (for classification) or mean squared error (for regression).
- **Internal Nodes:** Each internal node represents a decision based on a feature. For example, in a classification problem, it might represent a question like "Is the temperature above 30°C?".
- **Branches:** The branches emanating from each internal node represent the possible outcomes of the decision. In the temperature example, there would be two branches: one for "Yes" and another for "No."
- **Leaf Nodes:** The leaf nodes represent the final class labels or numerical values. For instance, in a binary classification problem, the leaf nodes might be labeled as "Class A" and "Class B.“
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

The main idea



- To make a prediction for a new data point, you start at the root node and traverse the tree by following the branches based on the values of the features. Ultimately, you reach a leaf node, which provides the predicted class label or regression value.

- How to Specify Test Condition?

  Depends on attribute types

  - Nominal
  - Ordinal
  - Continuous

  Depends on number of ways to split

  - 2-way split(binary) s. as (yes/no , true/false)
  - Multi-way split

Binary split:

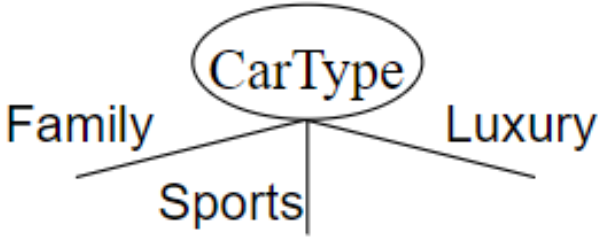
Divides values into two subsets. Need to find optimal partitioning.

Multi-way split:

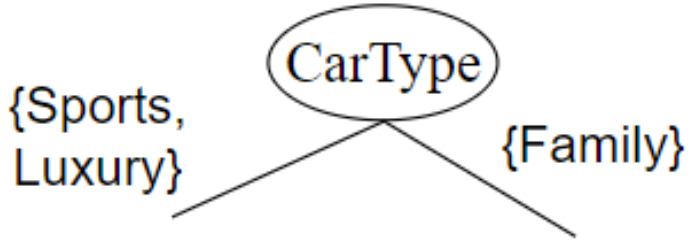
Use as many partitions as distinct values.

Splitting over nominal attributes

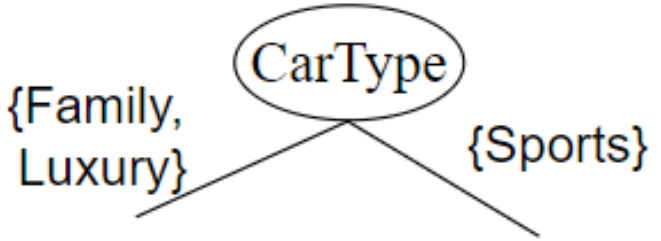
Mutli-way split : used as many partitions as distinct values



Binary split: divide the values into 2 subsets (requires finding the optimal partition)



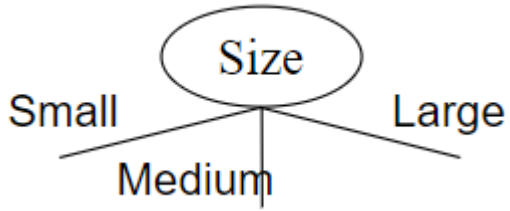
OR



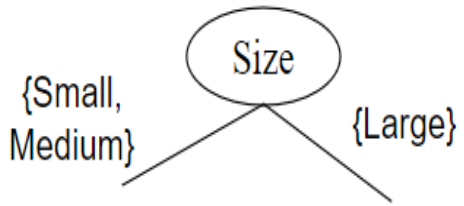
Grouping some features as one

Splitting over ordinal attributes

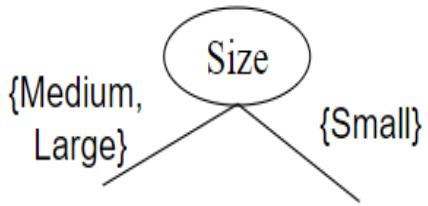
Mutli-way split : used as many partitions as distinct values

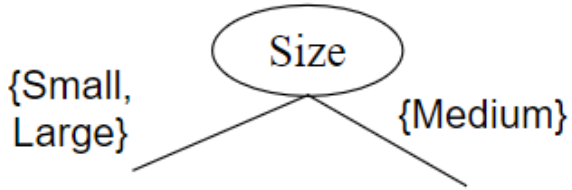


Binary split: divide the values into 2 subsets (requires finding the optimal partition)



OR





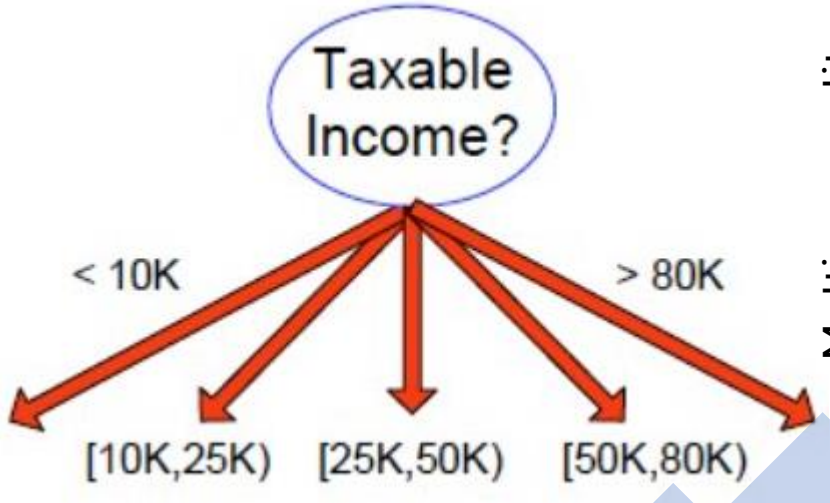
What about this split??

Splitting over continuous attributes

- **Binary Decision**: (A < v) or (A ≥ v)
  - consider all possible splits and finds the best cut
  - can be more compute intensive

Binary split

- **Discretization**: to form an ordinal categorical attribute ranges can be found by equal interval bucketing, equal frequency bucketing (percentiles), or clustering.

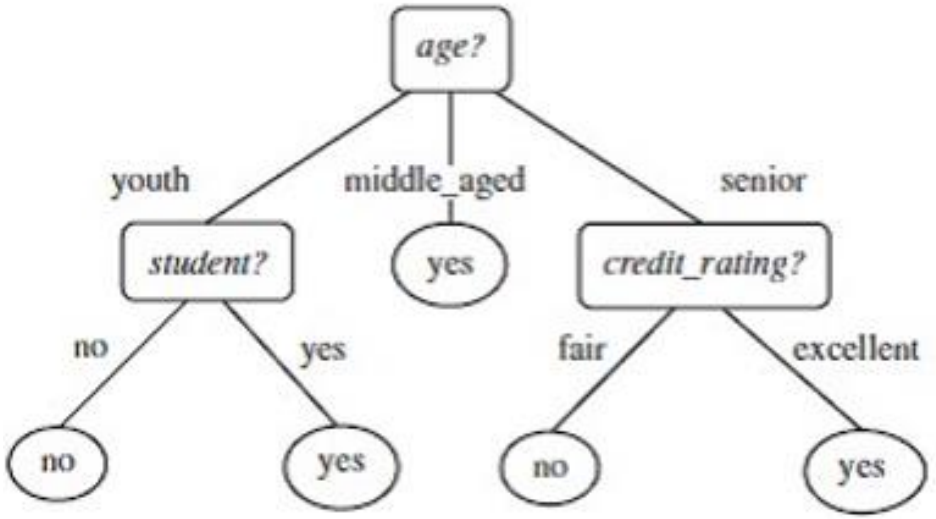


Multi- way split

Decision induction

- Determine how to split the records
  - How to specify the attribute test condition?
  - How to determine the best split?
  - Determine when to stop splitting

This depends on the nature of data



- Split the records based on an attribute test that optimizes certain criterion.

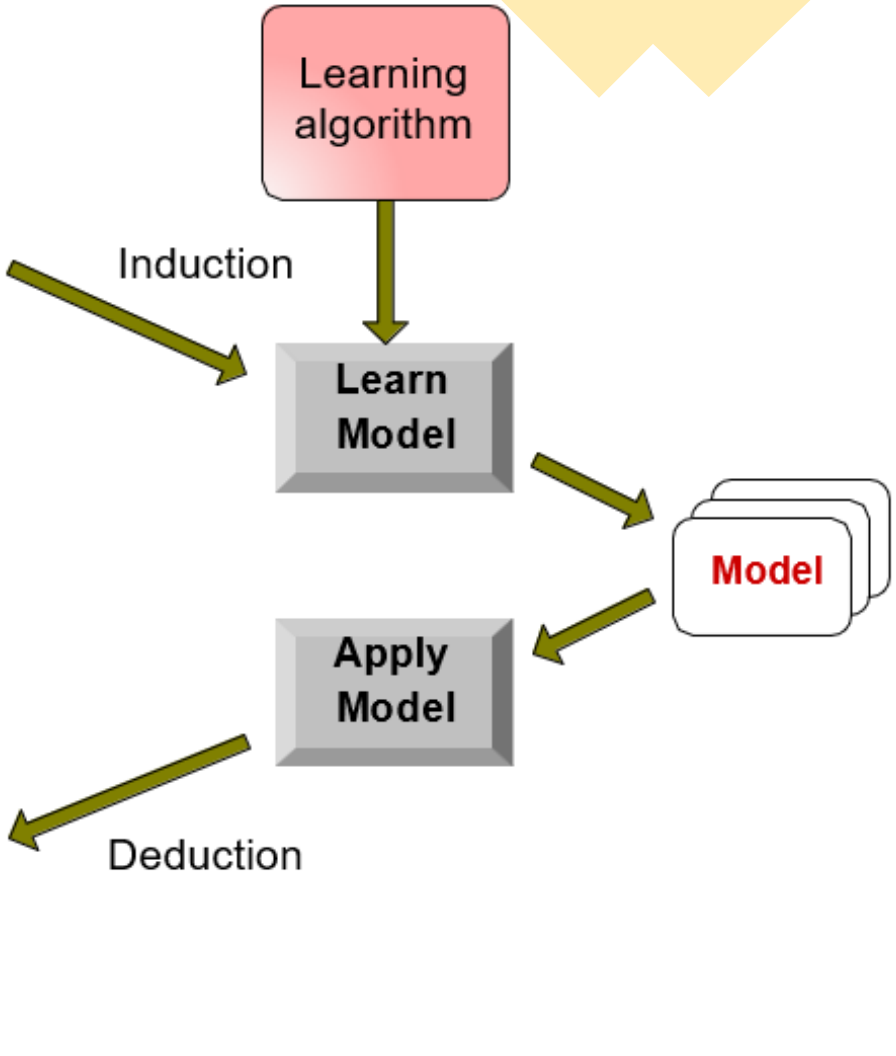
Classification process

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



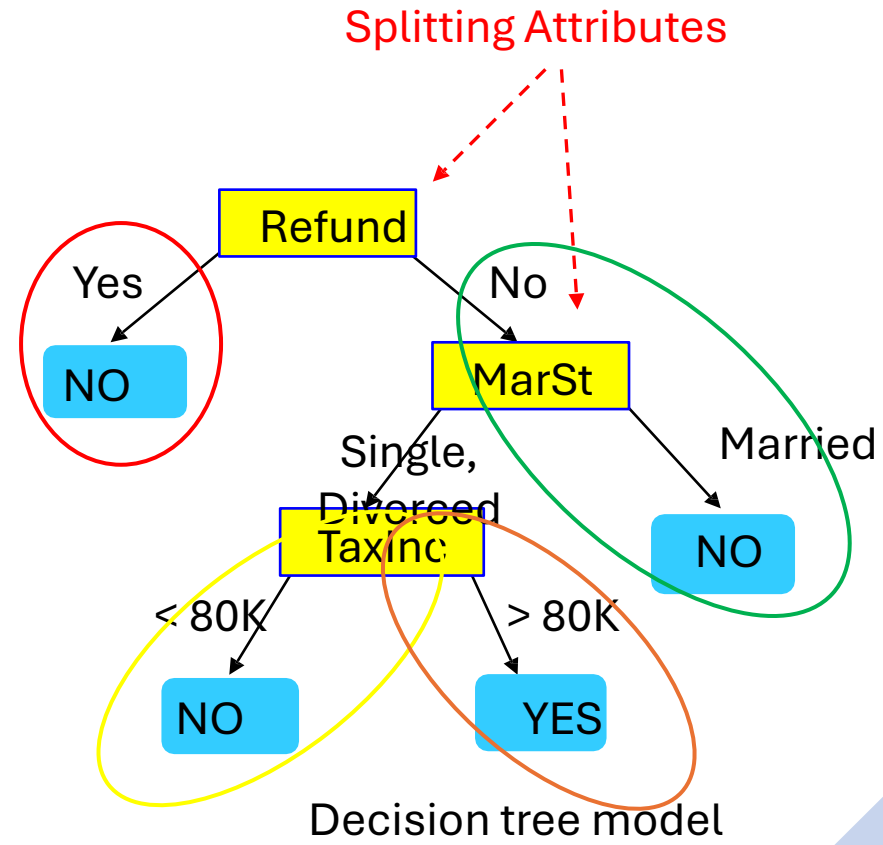
An illustrative example

- Start with **Refund** assume single and divorced can be classified as one class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

categorical categorical continuous class

Training data



Decision tree model

An illustrative example

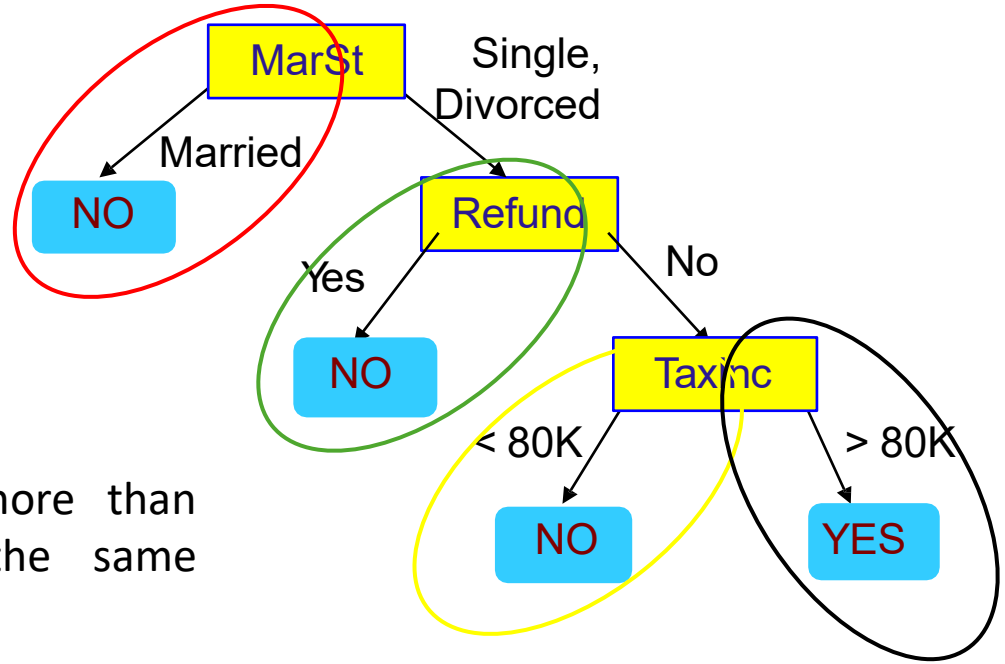
categorical categorical continuous class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training data

- Start with **Marital status** assume single and divorced can be classified as one class

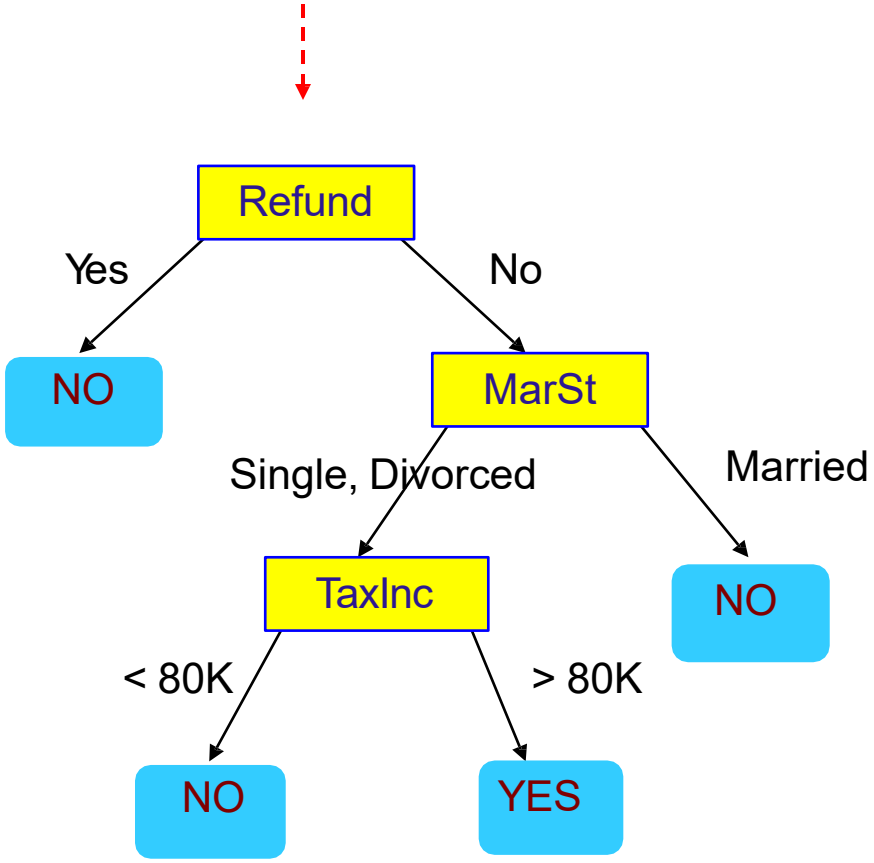
There could be more than one tree that fits the same data!



Decision tree model

Apply the model on test data

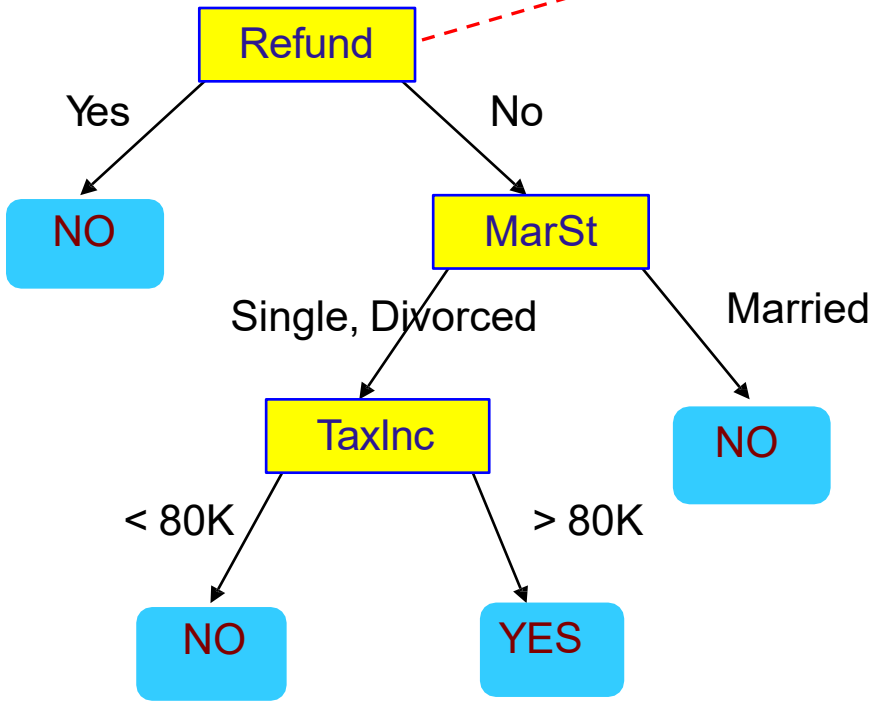
Start from the root of tree.



Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

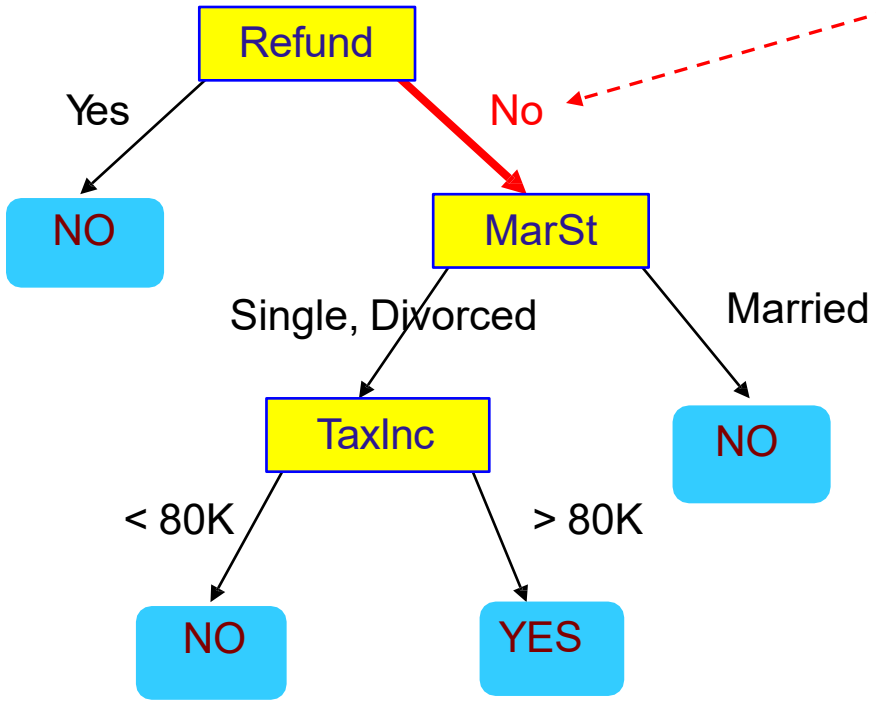
Apply the model on test data Cont.



Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Test Data

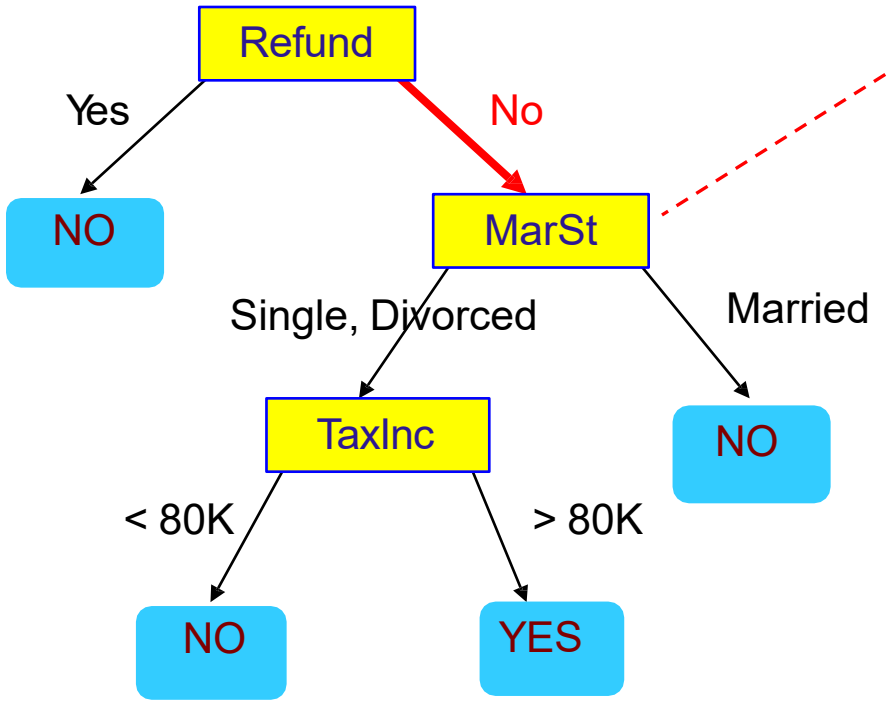
Apply the model on test data Cont.



Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Test Data

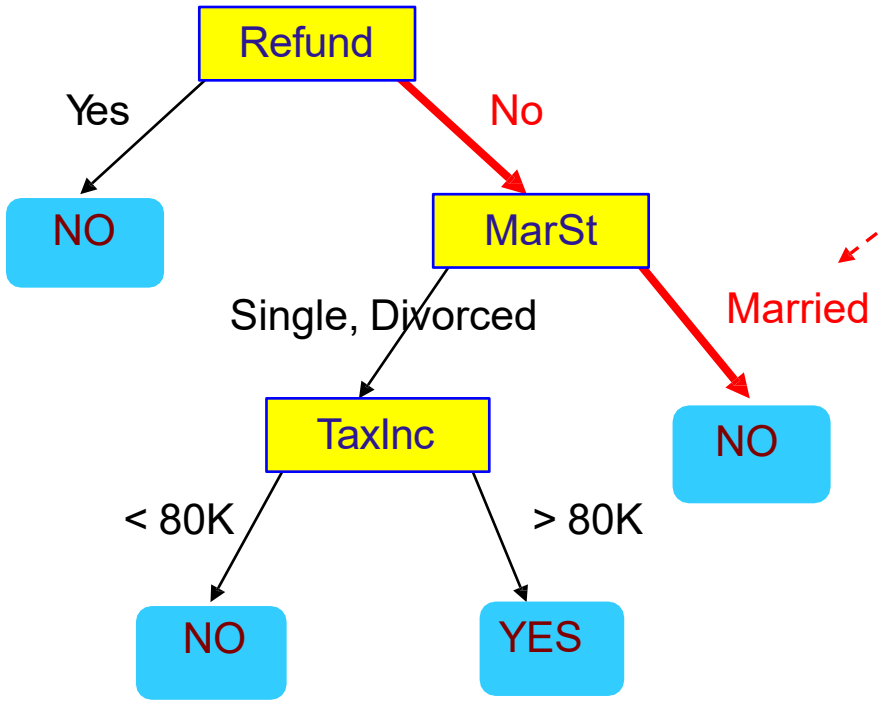
Apply the model on test data Cont.



Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Test Data

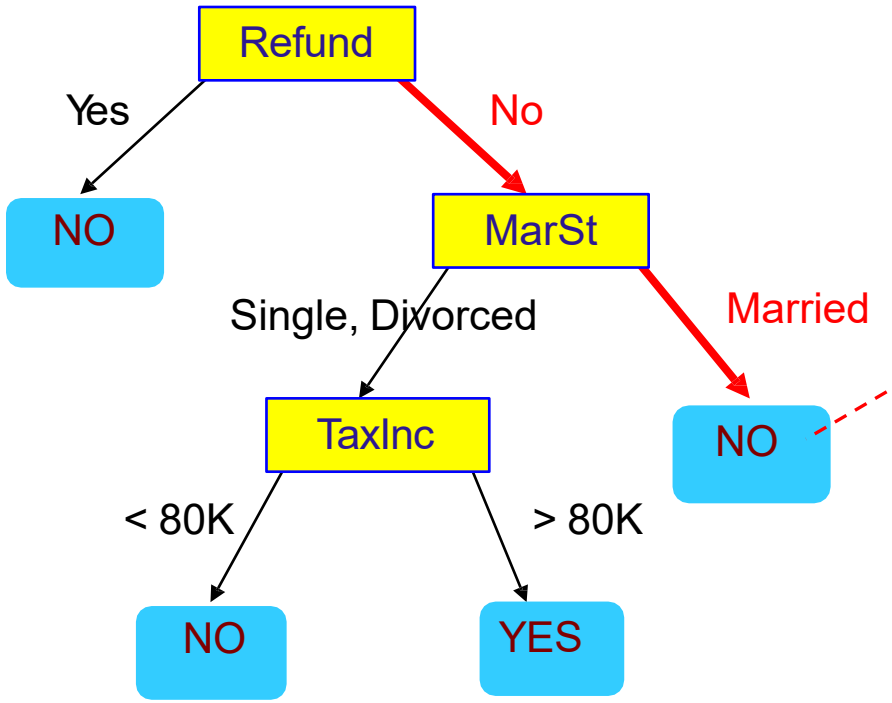
Apply the model on test data Cont.



Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Test Data

Apply the model on test data Cont.



Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Test Data

Assign Cheat to “No”

Decision Tree algorithm

1. Step-1: Begin the tree with the root node, says S, which contains the complete dataset.
2. Step-2: Find the best attribute in the dataset using Attribute Selection Measure(ASM).
3. Step-3: Divide the S into subsets that contains possible values for the best attributes.
4. Step-4: Generate the decision tree node, which contains the best attribute.
5. Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

How to select the best attribute for the root node and for sub-nodes. There are two popular techniques for ASM (Information Gain, Gini Index)

Building a DT

- **Tree Construction (basic idea)**

1. Pick an attribute for division of given data
2. Divide the given data into sets based on this attribute
3. For every set created above - repeat 1 and 2 until you find leaf nodes in all the branches of the tree - Terminate

Is a DT unique ?? →

1. The answer is No. We could generate many trees.
2. Our goal is to find a simpler tree which is easy to understand and as accurate as possible.
3. To get this tree, we need to reduce the impurity or uncertainty as much as possible.
4. We say that a subset of data is pure if all instances are belonging to the same class.
5. Thus, we need to maximize the information gain or gain ratio based on information theory.

Attribute Selection Measure (ASM) techniques

Information Gain:

- Information gain is the measurement of changes in **entropy** (i.e., it is a metric to measure the impurity in a given attribute. It specifies randomness in data) after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

  Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

Gini Index:

- Gini index is a measure of **impurity or purity** used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula: Gini Index= $1-\sum_j p_j{}^2$

Information Gain

Calculating entropy for a decision tree is a common step in the process of building and evaluating decision trees, especially in the context of machine learning and data analysis. Entropy in this context is used to measure the impurity or disorder of a dataset, which helps in deciding how to split the data into branches in the decision tree. Here's a step-by-step guide on how to calculate entropy for a decision tree:

Step 1: Understand the Problem
Before calculating entropy, you should have a clear understanding of your dataset and the specific decision you want to make. Decision trees are typically used for classification problems, so you need to know the classes or categories you are trying to predict.

Information Gain

Step 2: Calculate the Total Entropy (Entropy before Splitting)
The total entropy of a dataset is calculated using the following formula:

$$\text{Entropy}(S) = -p_1 log_2(p_1) - p_2 log_2(p_2) - \cdots - p_n log_2(p_n)$$

Where:

- **Entropy(S)** is the entropy of the dataset.
- $p_1, p_2, \ldots, p_n$ are the proportions of instances in each class in the dataset.
- For a binary classification problem with two classes (e.g., “yes” and “no”), you'll have p_1 representing the proportion of instances in class “yes” and p_2 representing the proportion of instances in class “no.”

Information Gain

Step 3: Calculate the Entropy after Splitting
Next, you'll calculate the entropy for each branch (child node) of the decision tree after a particular split. To do this, follow these steps:

a. For each branch, calculate the proportion (p_i) of instances belonging to each class within that branch.

b. Calculate the entropy of each branch using the same entropy formula as in Step 2, but now using the proportions p_i calculated in Step 3a.

c. Weight the entropy of each branch by the proportion of instances in that branch and sum them up to get the weighted entropy of the split.

Information Gain

Step 4: Calculate Information Gain
Information Gain is a measure of how much the entropy decreases after a split. It's calculated as follows:

$$\text{Information } Gain = Entropy(S) - Weighted\ Entropy\ after\ Split$$

A higher Information Gain indicates a better split, as it reduces the overall entropy and increases the purity of the branches.

Step 5: Repeat Steps 3 and 4 for All Possible Splits
You'll need to consider different features and thresholds (for continuous features) to determine the best split for your decision tree. Calculate the Information Gain for each potential split and choose the one with the highest Information Gain as the root of your decision tree or as the next split in the tree.

Repeat this process recursively for each branch until you reach a stopping criterion, such as a predefined tree depth or a minimum number of instances in a leaf node.

Predictors

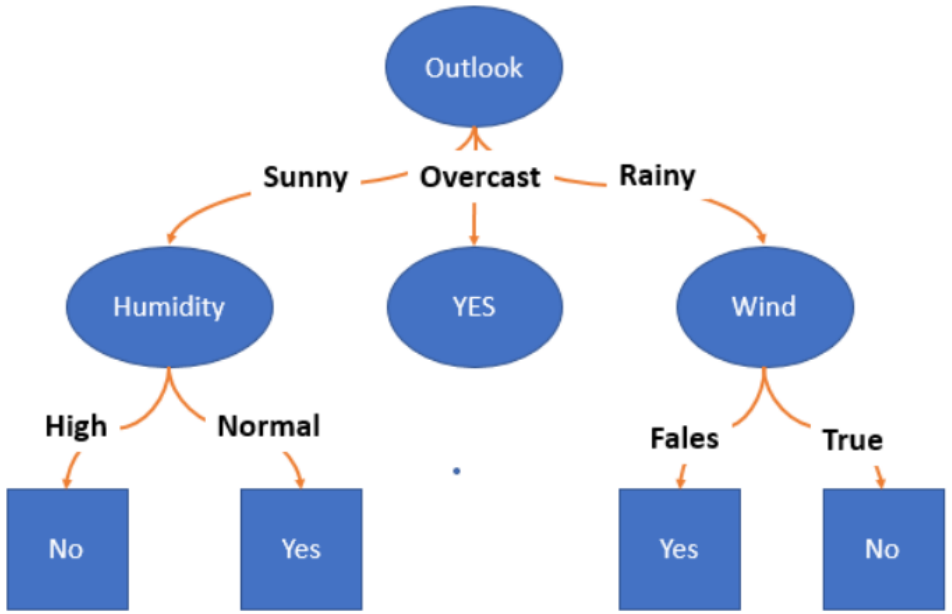
Target

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Check the complete example from here

Example

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



decision tree

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Attribute : Outlook

Values (outlook) = Sunny, Overcast, Rain

First, we need to calculate Entropy for our dependent/target/predicted variable. In our data set we have 9 YES and 5 NO out of 14 observations.

$$E(S) = \sum_{i=j}^{j+1} -p_i \log_2 p_i$$

$$p_i = -\frac{Number\ of\ occurence\ of\ either\ yes\ or\ no}{total\ nmber\ of\ count}$$

$$j = classes\ like\ Yes, No, etc\ \ldots$$

$S = [9+, 5-]$

$$Entropy(S) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.49$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Second: We need to calculate information gain for each and every variable., which ever variable is giving high value we will consider it as root node.

we need to group the data based on the categorical values (sunny, overcast, rain)

	Count of YES/No		
Outlook	**YES**	**No**	**Count of each group**
sunny	2	3	5
overcast	4	0	4
rainy	3	2	5

Then the Weight of Evidence or Information or Average Entropy can be calculated as follows.

$$-\frac{Count\ of\ YES}{count\ of\ group} * log(\frac{Count\ of\ YES}{Count\ of\ group}) - \frac{Count\ of\ No}{Count\ of\ group} * log(\frac{Count\ of\ No}{count\ of\ group})$$

$S_{Sunny} \leftarrow [2+, 3-]$ $\quad Entropy(S_{Sunny}) = -\frac{2}{5}log_2\frac{2}{5} - \frac{3}{5}log_2\frac{3}{5} = 0.971$

$S_{Overcast} \leftarrow [4+, 0-]$ $\quad Entropy(S_{Overcast}) = -\frac{4}{4}log_2\frac{4}{4} - \frac{0}{4}log_2\frac{0}{4} = 0$

$S_{Rain} \leftarrow [3+, 2-]$ $\quad Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.971$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Third: We need to calculate gain as follows

$$Gain\ (S, Outlook) = Entropy(S) - \sum_{v \in \{Sunny, Overcast, Rain\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Outlook) =$$

$$Entropy(S) - \frac{5}{14} Entropy(S_{Sunny}) - \frac{4}{14} Entropy(S_{Overcast}) - \frac{5}{14} Entropy(S_{Rain})$$

$$Gain(S, Outlook) = 0.94 - \frac{5}{14} 0.971 - \frac{4}{14} 0 - \frac{5}{14} 0.971 = 0.2464$$

Now we got the Gain value for Outlook is 0.24

similarly calculate for temperature, humidity and wind

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- The information gain for **Temperature** is **low (0.0289)**.
- Therefore, **Temperature is a weak splitting attribute** compared to others.

Attribute: Temperature

Values (Temp) = {Hot, Mild, Cool}

S=[9+,5−]

$$Entropy(S) = -\frac{9}{14}log_2\frac{9}{14} - \frac{5}{14}log_2\frac{5}{14} = 0.94$$

S_{Hot} =[2+,2−]

$$Entropy(S_{Hot}) = -\frac{2}{4}log_2\frac{2}{4} - \frac{2}{4}log_2\frac{2}{4} = 1.0$$

S_{Mild} =[4+,2−]

$$Entropy(S_{Mild}) = -\frac{4}{6}log_2\frac{4}{6} - \frac{2}{6}log_2\frac{2}{6} = 0.9183$$

S_{Cool} =[3+,1−]

$$Entropy(S_{Cool}) = -\frac{3}{4}log_2\frac{3}{4} - \frac{1}{4}log_2\frac{1}{4} = 0.8113$$

$$Gain(S, Temp) = Entropy(S) - \sum_{v\in\{Hot,Mild,Cool\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp) = 0.94 - \frac{4}{14}(1.0) - \frac{6}{14}(0.9183) - \frac{4}{14}(0.8113)$$

$$Gain(S, Temp) = 0.0289$$

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- The information gain of **Humidity (0.151)** is **higher than Temperature**.
- Therefore, **Humidity is a better splitting attribute**.

Attribute: Humidity

Values (Humidity) = {High, Normal}

S=[9+,5−] $\quad Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$

S_{High} =[3+,4−] $\quad Entropy(S_{High}) = -\frac{3}{7} log_2 \frac{3}{7} - \frac{4}{7} log_2 \frac{4}{7} = 0.9852$

S_{Normal} =[6+,1−] $\quad Entropy(S_{Normal}) = -\frac{6}{7} log_2 \frac{6}{7} - \frac{1}{7} log_2 \frac{1}{7} = 0.5916$

$$Gain(S, Humidity) = Entropy(S) - \sum_{v \in \{High, Normal\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Humidity) = 0.94 - \frac{7}{14}(0.9852) - \frac{7}{14}(0.5916)$$

$$Gain(S, Humidity) = 0.151$$

Attribute: Wind

Values (Wind) = {Strong, Weak}

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

S=[9+,5−]

$$Entropy(S) = -\frac{9}{14} log_2 \frac{9}{14} - \frac{5}{14} log_2 \frac{5}{14} = 0.94$$

S_{Strong} =[3+,3−]

$$Entropy(S_{Strong}) = -\frac{3}{6} log_2 \frac{3}{6} - \frac{3}{6} log_2 \frac{3}{6} = 1.0$$

S_{Weak} =[6+,2−]

$$Entropy(S_{Weak}) = -\frac{6}{8} log_2 \frac{6}{8} - \frac{2}{8} log_2 \frac{2}{8} = 0.8113$$

$$Gain(S, Wind) = Entropy(S) - \sum_{v \in \{Strong, Weak\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Wind) = 0.94 - \frac{6}{14}(1.0) - \frac{8}{14}(0.8113)$$

$$Gain(S, Wind) = 0.0478$$

- The information gain of **Wind (0.0478)** is **greater than Temperature**, but **less than Humidity**.
- Therefore, **Wind is a moderate splitting attribute**, but not the best.

Day	Outlook	Temp	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

$$Gain(S, Outlook) = 0.2464$$

$$Gain(S, Temp) = 0.0289$$

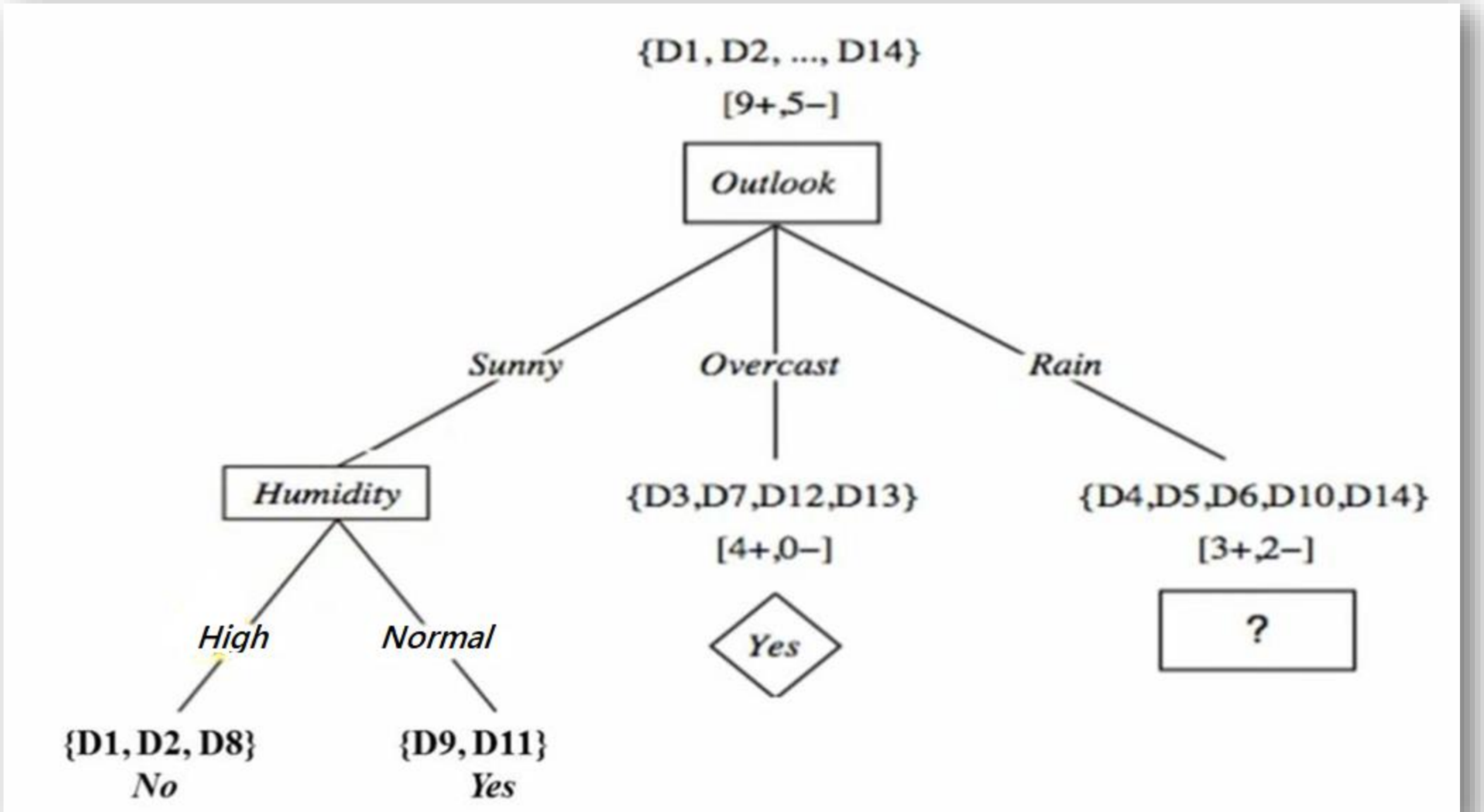
$$Gain(S, Humidity) = 0.1516$$

$$Gain(S, Wind) = 0.0478$$

- The attribute with the **highest information gain** is:

$$\boxed{Outlook\ (0.2464)}$$

➞Therefore, **Outlook is chosen as the root node** of the decision tree.


{D1, D2, ..., D14}
[9+,5−]
Outlook
Sunny
Overcast
Rain
Humidity
{D3,D7,D12,D13}
[4+,0−]
{D4,D5,D6,D10,D14}
[3+,2−]
High
Normal
Yes
?
{D1, D2, D8}
No
{D9, D11}
Yes

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

- The information gain of **Temperature (0.0192)** for the **Rain subset** is **very low**.
- Therefore, **Temperature is not a good splitting attribute** for this branch.
- This confirms why **Wind** was selected instead.

Attribute: Temperature (for Outlook = Rain subset)
Values (Temp) = {Hot, Mild, Cool}

Outlook = Rain

$$S_{Rain} = [3^+, 2^-]$$

$$Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$$

S_{Hot} =[0+,0−]	$Entropy(S_{Hot}) = 0.0$
S_{Mild} =[2+,1−]	$Entropy(S_{Mild}) = -\frac{2}{3}log_2\frac{2}{3} - \frac{1}{3}log_2\frac{1}{3} = 0.9183$
S_{Cool} =[1+,1−]	$Entropy(S_{Cool}) = -\frac{1}{2}log_2\frac{1}{2} - \frac{1}{2}log_2\frac{1}{2} = 1.0$

$$Gain(S_{Rain}, Temp) = Entropy(S_{Rain}) - \sum_{v\in\{Hot,Mild,Cool\}} \frac{|S_v|}{|S_{Rain}|} Entropy(S_v)$$

$$Gain(S_{Rain}, Temp) = 0.97 - \frac{0}{5}(0.0) - \frac{3}{5}(0.9183) - \frac{2}{5}(1.0)$$

$$Gain(S_{Rain}, Temp) = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

- The information gain of **Humidity (0.0192)** for the **Rain subset** is **very low**.
- Therefore, **Humidity is not a good splitting attribute** at this node.
- This confirms why **Wind** is chosen as the splitting attribute for the Rain branch.

Attribute: Humidity (for Outlook = Rain subset)
Values (Humidity) = {High, Normal}

Outlook = Rain

$$S_{Rain} = [3^+, 2^-]$$

$$Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$$

S_{High} =[1+,1−] $Entropy(S_{High}) = -\frac{1}{2}log_2\frac{1}{2} - \frac{1}{2}log_2\frac{1}{2} = 1.0$

S_{Normal} =[2+,1−] $Entropy(S_{Normal}) = -\frac{2}{3}log_2\frac{2}{3} - \frac{1}{3}log_2\frac{1}{3} = 0.9183$

$$Gain(S_{Rain}, Humidity) = Entropy(S_{Rain}) - \sum_{v\in\{High,Normal\}} \frac{|S_v|}{|S_{Rain}|} Entropy(S_v)$$

$$Gain(S_{Rain}, Humidity) = 0.97 - \frac{2}{5}(1.0) - \frac{3}{5}(0.9183)$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

- The information gain of **Wind (0.97)** for the **Rain subset** is **maximal**.
- Both resulting subsets are **pure** (entropy = 0).
- Therefore, **Wind is the best splitting attribute** for the Rain branch.

Attribute: Wind (for Outlook = Rain subset)
Values (Wind) = {Strong, Weak}

Outlook = Rain

$$S_{Rain} = [3^+, 2^-]$$

$$Entropy(S_{Rain}) = -\frac{3}{5}log_2\frac{3}{5} - \frac{2}{5}log_2\frac{2}{5} = 0.97$$

S_{Strong} =[0+,2−] $\quad Entropy(S_{Strong}) = 0.0$

S_{Weak} =[3+,0−] $\quad Entropy(S_{Weak}) = 0.0$

$$Gain(S_{Rain}, Wind) = Entropy(S_{Rain}) - \sum_{v\in\{Strong,Weak\}} \frac{|S_v|}{|S_{Rain}|} Entropy(S_v)$$

$$Gain(S_{Rain}, Wind) = 0.97 - \frac{2}{5}(0.0) - \frac{3}{5}(0.0)$$

$$Gain(S_{Rain}, Wind) = 0.97$$

Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
Dl0	Mild	Normal	Weak	Yes
Dl4	Mild	High	Strong	No

$$Gain(S_{Rain}, Temp) = 0.0192$$

$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$

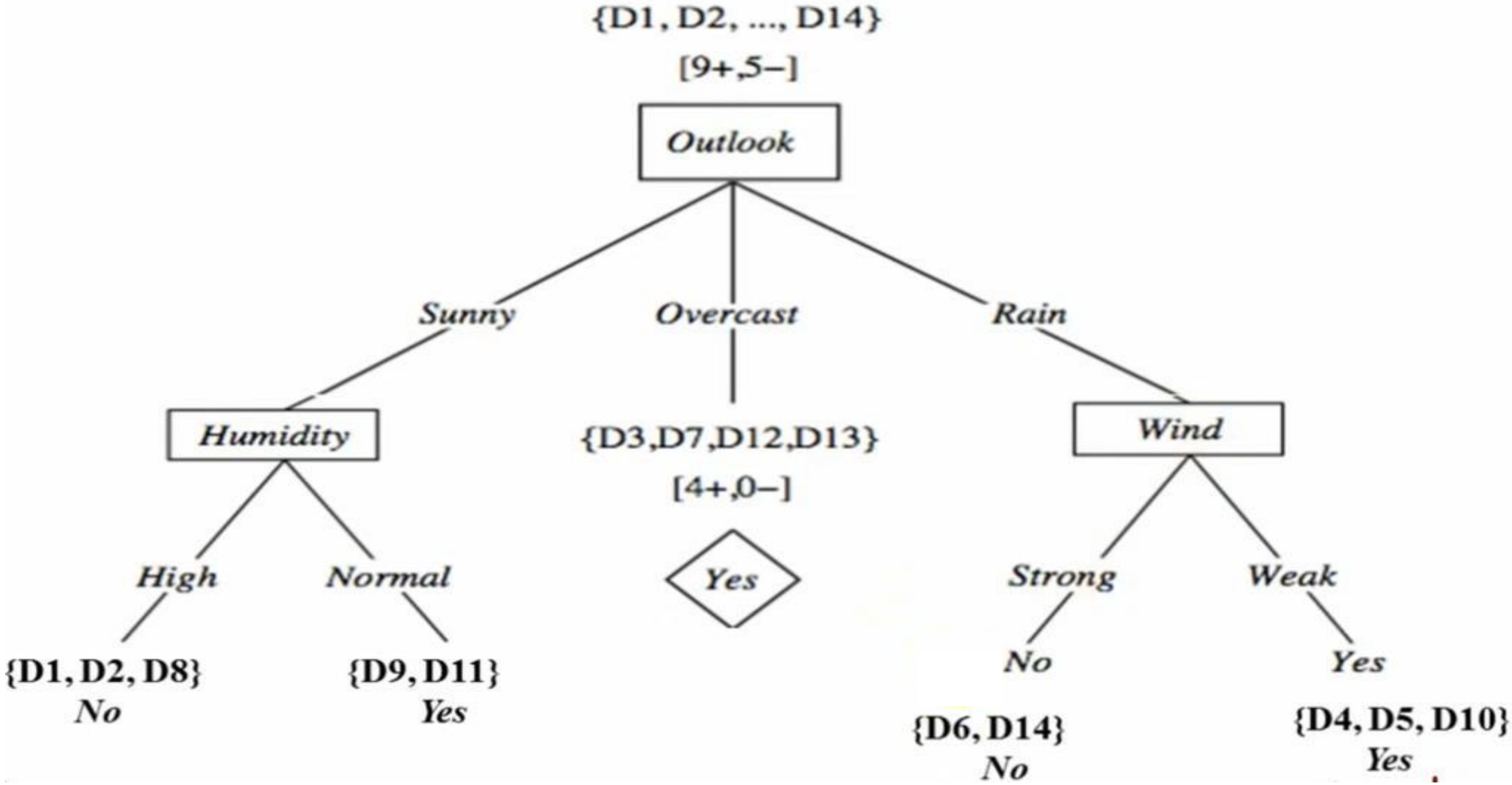
Day	Temp	Humidity	Wind	Play Tennis
D4	Mild	High	Weak	Yes
D5	Cool	Normal	Weak	Yes
D6	Cool	Normal	Strong	No
D10	Mild	Normal	Weak	Yes
D14	Mild	High	Strong	No

$$Gain(S_{Rain}, Temp) = 0.0192$$

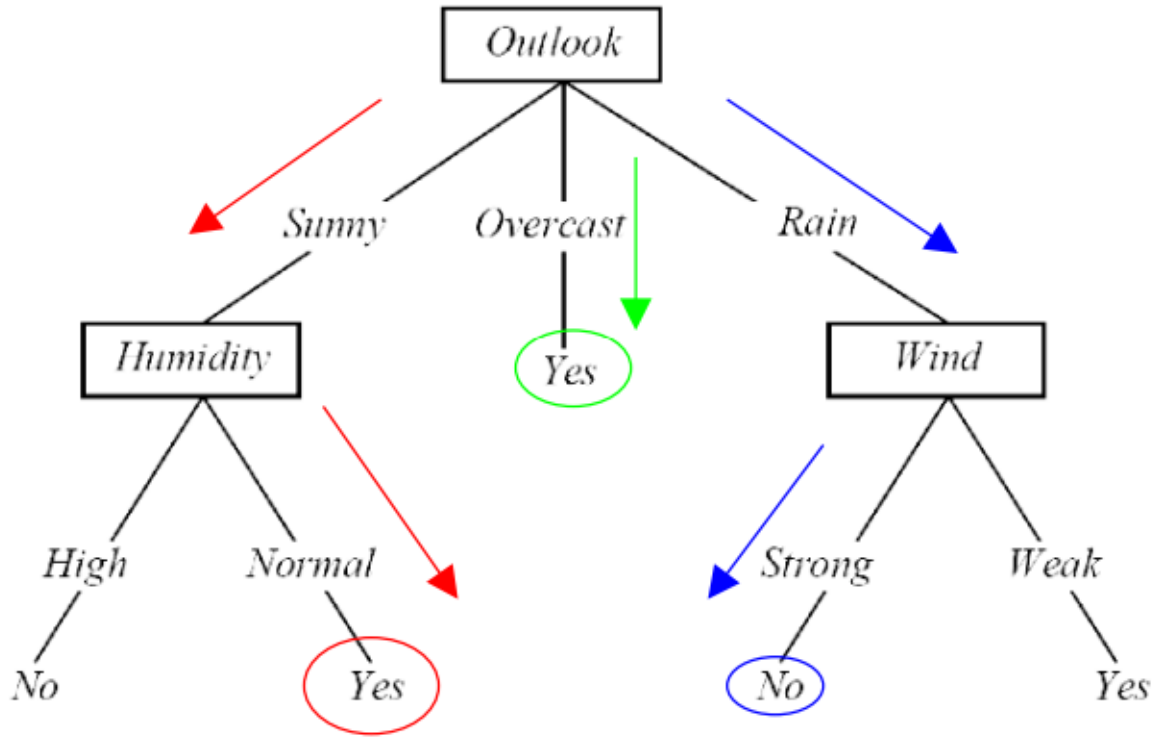
$$Gain(S_{Rain}, Humidity) = 0.0192$$

$$Gain(S_{Rain}, Wind) = 0.97$$

- The attribute with the **highest information gain** is **Wind (0.97)**.
- Therefore, **Wind is selected as the splitting attribute** for the **Outlook = Rain** branch.


{D1, D2, ..., D14}
[9+,5−]
Outlook
Sunny
Overcast
Rain
Humidity
{D3,D7,D12,D13}
[4+,0−]
Wind
High
Normal
Yes
Strong
Weak
{D1, D2, D8}
No
{D9, D11}
Yes
No
Yes
{D6, D14}
No
{D4, D5, D10}
Yes

Each path from the root of the DT to a leaf can be interpreted as a decision rule.



IF *Outlook* = *Sunny* AND *Humidity* = *Normal* THEN *Playtennis* = *Yes*

IF *Outlook* = *Overcast* THEN *Playtennis* = *Yes*

IF *Outlook* = *Rain* AND *Wind* = *Strong* THEN *Playtennis* = *No*

Gini index

Step 1: Understand the Problem

Before calculating entropy, you should have a clear understanding of your dataset and the specific decision you want to make. Decision trees are typically used for classification problems, so you need to know the classes or categories you are trying to predict.

Step 2: Calculate the Total Gini Impurity (Impurity before Splitting)

The total Gini impurity of a dataset is calculated using the following formula: $Gini(S) = 1 - \sum_{i=1}^{n} (p_i)^2$

Where:

- **Gini(S)** is the Gini impurity of the dataset.
- p_i is the proportion of instances belonging to class i in the dataset.
- n is the total number of classes.
- For a binary classification problem with two classes (e.g., "yes" and "no"), you'll have p_1 representing the proportion of instances in class "yes" and p_2 representing the proportion of instances in class "no."

Gini index

Step 3: Calculate the Gini Impurity after Splitting

Next, you'll calculate the Gini impurity for each branch (child node) of the decision tree after a particular split. To do this, follow these steps:

a. For each branch, calculate the proportion (p_i (of instances belonging to each class within that branch.
b. Calculate the Gini impurity of each branch using the same Gini impurity formula as in Step 2, but now using the proportions $p_i$calculated in Step 3a.
c. Weight the Gini impurity of each branch by the proportion of instances in that branch and sum them up to get the weighted Gini impurity of the split.

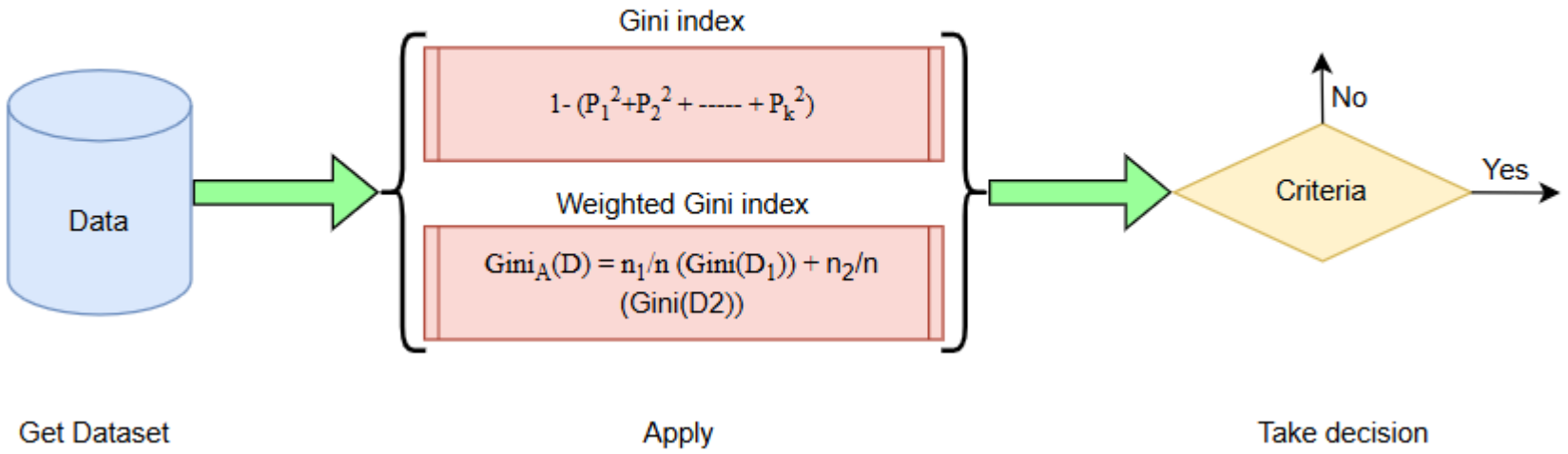
Step 4: Calculate Gini Gain (or Reduction in Gini Impurity)

Gini Gain is a measure of how much the Gini impurity decreases after a split. It is calculated as follows:

$$Gini\ Gain = Gini(S) - Weighted\ Gini\ Impurity\ after\ Split$$

A higher Gini Gain indicates a better split, as it reduces the overall impurity and increases the purity of the branches.

Construct decision Tree using Gini Index



- $p1, p_2, \ldots, p_k$ are the probabilities of each class in the node.

- The $Gini_A(D)$ represents the weighted Gini index for the entire dataset D. It's a measure of impurity or inequality in the dataset, considering the weighted average of the impurities of two subsets, D_1 and D_2.
  - n_1 :This is the number of instances (data points) in subset D_1.
  - n_2 :This is the number of instances (data points) in subset D_2.
  - n :The total number of instances in the entire dataset $D(n = n1 + n2)$.

$Gini(D_1)$:This is the Gini index of subset D_1 ,which quantifies the impurity or uncertainty of class labels in D1. A lower Gini index indicates higher purity.

$Gini(D_2)$:This is the Gini index of subset D_2 ,similar to $Gini(D_1)$,but for the other subset.

Multi-classification problem

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Check the complete example from here

Example

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Compute the **Gini Index** for the overall collection of training examples.

Given

There are **four possible output classes**: Cinema, Tennis, Stay In, and Shopping

The dataset contains **10 instances** in total:

- Cinema: 6 instances
- Tennis: 2 instances
- Stay In: 1 instance
- Shopping: 1 instance

Class Probabilities

$P(Cinema) = \frac{6}{10}$ $P(Tennis) = \frac{2}{10}$

$P(Stay\ In) = \frac{1}{10}$ $P(Shopping) = \frac{1}{10}$

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2$$

$$Gini(S) = 1 - \left[\left(\frac{6}{10}\right)^2 + \left(\frac{2}{10}\right)^2 + \left(\frac{1}{10}\right)^2 + \left(\frac{1}{10}\right)^2\right]$$

$$Gini(S) = 1 - (0.36 + 0.04 + 0.01 + 0.01)$$

$$\boxed{Gini(S) = 0.58}$$

- A Gini Index of **0.58** indicates **moderate impurity**.
- The dataset is **not pure**, as multiple classes are present.
- Lower Gini values indicate purer nodes; higher values indicate more mixed classes.

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Computation of Gini Index for Attribute: Money

Possible Values of Money

- **Rich** → 7 examples
- **Poor** → 3 examples

Total number of examples = **10**

Case 1: Money = Poor

All 3 examples belong to **Cinema**.

Class Distribution

- Cinema: 3
- Tennis: 0
- Stay In: 0
- Shopping: 0

This subset is **pure**.

$$Gini(S_{Poor}) = 1 - \left(\frac{3}{3}\right)^2 = 1 - 1 = 0$$

Case 2: Money = Rich

Class Distribution

- Tennis: 2
- Cinema: 3
- Stay In: 1
- Shopping: 1

Total = 7 examples

$$Gini(S_{Rich}) = 1 - \left[\left(\frac{2}{7}\right)^2 + \left(\frac{3}{7}\right)^2 + \left(\frac{1}{7}\right)^2 + \left(\frac{1}{7}\right)^2\right]$$

$$Gini(S_{Rich}) = 1 - (0.0816 + 0.1837 + 0.0204 + 0.0204)$$

$$\boxed{Gini(S_{Rich}) = 0.694}$$

Weighted Gini Index for Money

$$Gini(S, Money) = \frac{3}{10} \cdot Gini(S_{Poor}) + \frac{7}{10} \cdot Gini(S_{Rich})$$

$$Gini(S, Money) = \frac{3}{10}(0) + \frac{7}{10}(0.694)$$

$$\boxed{Gini(S, Money) = 0.486}$$

A **lower Gini value** indicates a better split.

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Computation of Gini Index for Attribute: Parents

Possible Values of Parents

- **Yes** → 5 examples
- **No** → 5 examples

Total number of examples = **10**

Case 1: Parents = Yes

All 5 examples belong to **Cinema**.

Class Distribution

- Cinema: 5
- Tennis: 0
- Stay In: 0
- Shopping: 0

This subset is **pure**.

$$Gini(S_{Yes}) = 1 - \left(\frac{5}{5}\right)^2 = 1 - 1 = 0$$

Case 2: Parents = No

Class Distribution

- Tennis: 2
- Stay In: 1
- Shopping: 1
- Cinema: 1

Total = 5 examples

$$Gini(S_{No}) = 1 - \left[\left(\frac{2}{5}\right)^2 + \left(\frac{1}{5}\right)^2 + \left(\frac{1}{5}\right)^2 + \left(\frac{1}{5}\right)^2\right]$$

$$Gini(S_{No}) = 1 - (0.16 + 0.04 + 0.04 + 0.04)$$

$$\boxed{Gini(S_{No}) = 0.72}$$

Weighted Gini Index for Parents

$$Gini(S, Parents) = \frac{5}{10} \cdot Gini(S_{Yes}) + \frac{5}{10} \cdot Gini(S_{No})$$

$$Gini(S, Parents) = \frac{5}{10}(0) + \frac{5}{10}(0.72)$$

$$\boxed{Gini(S, Parents) = 0.36}$$

A **lower Gini value** indicates a better split.

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Computation of Gini Index for Attribute: Weather

Possible Values of Weather

- **Sunny** → 3 examples
- **Rainy** → 3 examples
- **Windy** → 4 examples

Total number of examples = **10**

Weighted Gini Index for Parents

Case 1: Weather = Sunny

Class Distribution

- Cinema: 2
- Tennis: 1

$$Gini(S_{Sunny}) = 1 - \left[\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2\right]$$

$$Gini(S_{Sunny}) = 1 - (0.444 + 0.111) = \boxed{0.444}$$

Case 2: Weather = Rainy

Class Distribution

- Cinema: 2
- Stay In: 1

$$Gini(S_{Rainy}) = 1 - \left[\left(\frac{2}{3}\right)^2 + \left(\frac{1}{3}\right)^2\right]$$

$$Gini(S_{Rainy}) = 1 - (0.444 + 0.111) = \boxed{0.444}$$

Case 3: Weather = Windy

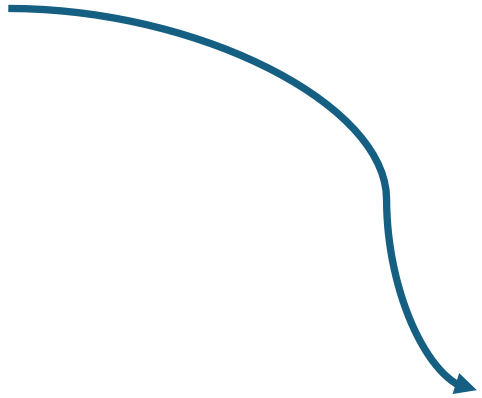
Class Distribution

- Cinema: 3
- Shopping: 1

$$Gini(S_{Windy}) = 1 - \left[\left(\frac{3}{4}\right)^2 + \left(\frac{1}{4}\right)^2\right]$$

$$Gini(S_{Windy}) = 1 - (0.5625 + 0.0625) = \boxed{0.375}$$

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis



Weighted Gini Index for Weather

$$Gini(S, Weather) = \frac{3}{10} \cdot Gini(S_{Sunny}) + \frac{3}{10} \cdot Gini(S_{Rainy}) + \frac{4}{10} \cdot Gini(S_{Windy})$$

$$Gini(S, Weather) = \frac{3}{10}(0.444) + \frac{3}{10}(0.444) + \frac{4}{10}(0.375)$$

$$Gini(S, Weather) = 0.1332 + 0.1332 + 0.15$$

$$\boxed{Gini(S, Weather) = 0.4164}$$

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Comparison of Gini Index Values

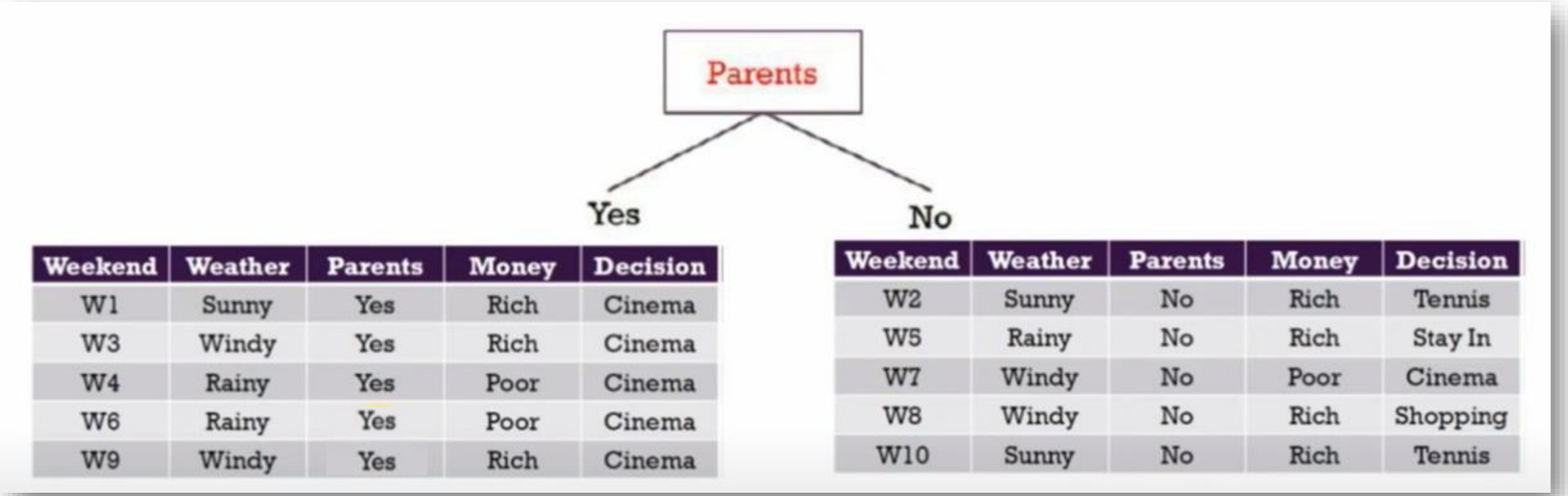
Attribute	Gini Index
Weather	0.416
Parents	**0.36**
Money	0.486

- The attribute with the **smallest Gini Index** is **Parents (0.36)**.
- Therefore, **Parents is selected as the splitting attribute** (root node).

Gini Gain

$$Gini\ Gain = Gini(S) - \text{Weighted Gini after split}$$

- Larger Gini Gain ⇔ Better split
- Since Parents has the **lowest weighted Gini**, it has the **highest Gini Gain**



Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W6	Rainy	Yes	Poor	Cinema
W9	Windy	Yes	Rich	Cinema

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Cinema

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Computation of Gini Index for (Parents = No | Weather Attribute)

- We compute the Gini impurity for the **subset where Parents = No**, considering the **Weather** attribute.

Weather = Windy (2 examples)

For **Parents = No** and **Weather = Windy**, the class distribution is:

- Cinema: 1
- Shopping: 1

$$Gini(S_{Windy}) = 1 - \left[\left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2\right]$$

$$Gini(S_{Windy}) = 1 - (0.25 + 0.25) = \boxed{0.5}$$

Weighted Gini Index for (Parents = No | Weather)

Assuming the Weather distribution under **Parents = No** is:

- Sunny → 2 examples (pure → Gini = 0)
- Rainy → 1 example (pure → Gini = 0)
- Windy → 2 examples (Gini = 0.5)

Total = 5 examples

Weighted Average Calculation

$$Weighted\ Gini(Parents = No \mid Weather) = 0 \times \frac{2}{5} + 0 \times \frac{1}{5} + 0.5 \times \frac{2}{5}$$

$$\boxed{Weighted\ Gini = 0.2}$$

This indicates a **good secondary split** under the Parents = No branch.

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Computation of Gini Index for (Parents = No | Money Attribute)

Money = Rich (4 examples)
For the subset where **Parents = No** and **Money = Rich**, the class distribution is:

- Stay In: 1
- Shopping: 1
- Tennis: 2

Total = 4 examples

$$Gini(S_{Rich}) = 1 - \left[\left(\frac{1}{4}\right)^2 + \left(\frac{1}{4}\right)^2 + \left(\frac{2}{4}\right)^2\right]$$

$$Gini(S_{Rich}) = 1 - (0.0625 + 0.0625 + 0.25)$$

$$\boxed{Gini(S_{Rich}) = 0.625}$$

- The subset **(Parents = No, Money = Rich)** contains **multiple classes**, so it is **impure**.
- The Gini Index value **0.625** indicates **high impurity**.
- Therefore, **Money is not a strong splitting attribute** under the **Parents = No** branch.

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Computation of Gini Index for (Parents = No | Money Attribute)

Money = Poor (1 example)
For the subset where **Parents = No** and **Money = Poor**, the class distribution is:

- Cinema: 1

Total = 1 example

$$Gini(S_{Poor}) = 1 - \left[\left(\frac{1}{1}\right)^2\right]$$

$$Gini(S_{Poor}) = 1 - 1 = \boxed{0}$$

This subset is **pure**.

Weighted Gini Index for (Parents = No | Money)
From previous calculation:

- **Money = Rich** → 4 examples, $Gini = 0.625$
- **Money = Poor** → 1 example, $Gini = 0$

Total examples under **Parents = No** = 5

Weighted Average Calculation

$$Weighted\ Gini(Parents = No \mid Money) = \frac{4}{5}(0.625) + \frac{1}{5}(0)$$

$$\boxed{Weighted\ Gini = 0.5}$$

Since **0.5 > 0.2 (Weather split)**, **Money is a worse splitting attribute** than Weather for this branch.

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Comparison of Gini Index Values (Parents = No)

$$Gini(Parents = No \mid Weather) = 0.2$$
$$Gini(Parents = No \mid Money) = 0.5$$

- The attribute with the **smallest Gini Index** is **Weather (0.2)**.
- Therefore, **Weather is selected as the splitting attribute** under the **Parents = No** branch.

Why Is Weather Selected?

- Weather produces **purer child nodes** compared to Money.
- Lower Gini Index ⇒ **less impurity after split**.

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Case Analysis: Parents = No

1 Parents = No & Weather = Sunny

From the dataset:

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W10	Sunny	No	Rich	Tennis

- All instances belong to the **same class: Tennis**
- This subset is **pure**

$$\boxed{Decision = Tennis}$$

No further splitting is required.

Parents = No & Weather = Windy

This subset **contains mixed class labels**, so it is **not pure**

- Since multiple decision classes exist,
- **Further splitting is required** on another attribute (e.g., Money).

Weekend	Weather	Parents	Money	Decision
W2	Sunny	No	Rich	Tennis
W5	Rainy	No	Rich	Stay In
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W10	Sunny	No	Rich	Tennis

Case Analysis: Parents = No

1 Parents = No & Weather = Rainy

From the dataset:

Weekend	Weather	Parents	Money	Decision
W5	Rainy	No	Rich	Stay In

- All instances belong to **Stay In**
- Only **one class** exists

$$Decision = Stay\ In$$

This node is **pure**, so **no further splitting** is required.

2 Parents = No & Weather = Windy

Weekend	Weather	Parents	Money	Decision
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping

- Multiple classes exist (**Cinema** and **Shopping**)
- Node is **impure**

- **Further splitting is needed**
- Next attribute to test: **Money**

Weekend	Weather	Parents	Money	Decision
W1	Sunny	Yes	Rich	Cinema
W2	Sunny	No	Rich	Tennis
W3	Windy	Yes	Rich	Cinema
W4	Rainy	Yes	Poor	Cinema
W5	Rainy	No	Rich	Stay In
W6	Rainy	Yes	Poor	Cinema
W7	Windy	No	Poor	Cinema
W8	Windy	No	Rich	Shopping
W9	Windy	Yes	Rich	Cinema
W10	Sunny	No	Rich	Tennis

Final Classification Rules

1. IF Parents = Yes → **Cinema**
2. IF Parents = No AND Weather = Sunny → **Tennis**
3. IF Parents = No AND Weather = Rainy → **Stay In**
4. IF Parents = No AND Weather = Windy AND Money = Poor → **Cinema**
5. IF Parents = No AND Weather = Windy AND Money = Rich → **Shopping**

Decision Tree Classifier

Advantages:

- Easy to understand and generate rules.
- There are almost null hyper-parameters to be tuned.
- Complex Decision Tree models can be significantly simplified by its visualizations.

Disadvantages:

- Might suffer from overfitting.
- Does not easily work with non-numerical data.
- Low prediction accuracy for a dataset in comparison with other machine learning classification algorithms.
- When there are many class labels, calculations can be complex.

Impurity Criterion

Gini Index

$$I_G = 1 - \sum_{j=1}^{c} p_j^2$$

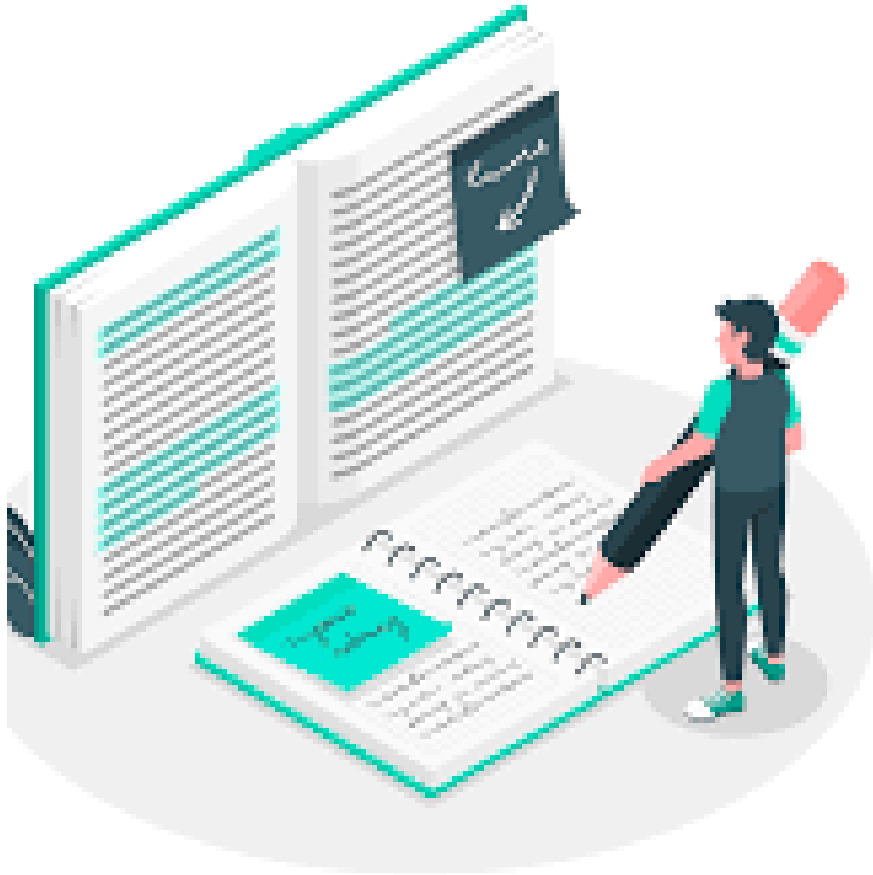
p_j: proportion of the samples that belongs to class c for a particular node

Entropy

$$I_H = -\sum_{j=1}^{c} p_j log_2(p_j)$$

p_j: proportion of the samples that belongs to class c for a particular node.

*This is the the definition of entropy for all non-empty classes ($p \neq 0$). The entropy is 0 if all samples at a node belong to the same class.



Test Your Knowledge

MCQ

In an SVM, *support vectors* are:
A) Points farthest from the hyperplane
B) Points closest to the hyperplane
C) Points with zero margin
D) Points always correctly classified

The parameter **C** in a soft-margin SVM controls:
A) The number of support vectors
B) The number of kernel functions used
C) The trade-off between margin width and classification error
D) The bias term in the model

The **Gini Index** in a Decision Tree measures:
A) Purity or impurity of a split
B) Correlation between features
C) The distance between branches
D) Model accuracy

Essay

Q: What is the role of the **kernel function** in an SVM?

A: It transforms data into a higher-dimensional space where classes become linearly separable.

الرواد الرقميون
DiGiLiANS

وزارة الاتصالات
وتكنولوجيا المعلومات

THANK you